Fully Secure DKG Protocols for Discrete Logarithm Revisited

Karim Baghery and Hossein Moghaddas

COSIC, KU Leuven, Leuven, Belgium
firstname.lastname@kuleuven.be
January 11, 2026

Abstract. In EUROCRYPT 1999, Gennaro, Jarecki, Krawczyk, and Rabin (GJKR) showed that in the well-known Pedersen robust Distributed Key Generation (DKG) protocol for the Discrete Logarithm (DL), an adversary can bias the distribution of the resulting public key. To address this issue, they proposed a fully secure, statistically unbiased variant of the Pedersen DKG protocol. The GJKR protocol achieves robustness and guarantees that the final public key remains uniformly random, even in the presence of computationally unbounded corrupted parties, though at the cost of $O(n^2)$ computational complexity, where n denotes the number of parties. In this paper, we revisit fully secure robust DKG protocols for the DL setting and propose three more efficient alternatives, each achieving $O(n)$ computational complexity while offering different trade-offs in security, efficiency, and round complexity. Our first protocol, like the GJKR scheme, guarantees that the distribution of the final public key remains uniformly random, even against computationally unbounded adversaries. The second protocol is concretely more efficient and ensures that the public key distribution is *computationally* indistinguishable from uniform. In our third construction, we focus on minimizing the number of rounds in the second protocol and present a 3-round variant of it. Our third scheme can be viewed as a fully secure and round-reduced variant of the biased construction by Atapoor et al. (ASIACRYPT 2023). In comparison with the most recent low-round fully secure DKG protocols by Katz (CRYPTO 2024), Cascudo-David (EUROCRYPT 2024), Kate et al. (CCS 2024), and Boneh et al. (EUROCRYPT 2025)–all of which achieve three rounds via two online rounds and one preprocessing round (or vice versa)–our three-round DKG protocol requires only $O(n)$ exponentiations, as opposed to at least $O(n^2)$.

Keywords: Fully Secure Distributed Key Generation · Discrete Logarithm · Shamir Secret Sharing · Robust DKG.

1 Introduction

During the last two decades, significant strides have been made in the development of Distributed Key Generation (DKG) protocols tailored for Discrete Logarithm (DL) instances. These protocols have emerged as crucial components

in threshold cryptography, enabling secure key generation, signing, and decryption across distributed parties. In a DKG protocol, each party uses a secret sharing scheme to share their secret among other parties, allowing only qualified set of them to reconstruct the shared secret.

Traditional secret sharing schemes, like Shamir's protocol [19], assume the presence of honest dealer and parties. To mitigate this assumption, Verifiable Secret Sharing (VSS) schemes [10, 11] have been developed that withstand various attacks, including incorrect share distribution and share reconstruction. Numerous VSS schemes are build on regular secret sharing protocols, adding verifiability features on top [1, 3, 11, 17]. Many of known VSS schemes like Feldman [11] and Pedersen [17] use Shamir secret sharing and exploit the homomorphic property of the Discrete Logarithm (DL) and Pedersen commitment to achieve verifiability. In [1], Atapoor, Baghery, Cozzo, and Pedersen (ABCP) introduced the first practical hash-based VSS scheme in the honest majority setting which uses a quantum Random Oracle (RO) and a (collapsing) hash-based commitment. In [8], Cascudo, Cozzo, and Giunta demonstrated that, by allowing a logarithmic increase in private communication, the broadcast overhead in the ABCP scheme can be reduced in the optimistic case. More recently, Baghery [3] generalized the ABCP [1] scheme and proposed a unified framework, dubbed Π, enabling the construction of Shamir-based VSS schemes using both homomorphic and non-homomorphic commitments in both classic and quantum RO models. Notably, instantiation of Π with different commitment schemes has yielded various practical VSS schemes [3]. Particularly, the instantiation employing hash-based commitments has yielded the state-of-the-art VSS scheme, showcasing a consistent improvement across all metrics when compared to the hash-based VSS scheme proposed by ABCP [1]. Recently, Baghery, Bardeh, Khazaei, and Rahimi (BBKR) [4] presented a round-optimal (i.e., 2-round) variant of the unified framework Π [3], so called 2R-Π(2-Round Π), which allows one to construct practical round-optimal VSS schemes in the honest majority setting.

In a DKG protocol for DL, the objective for n parties is to generate a public key $y = g^x$ in a fully distributed manner, where g represents a fixed generator of a cyclic group $\mathbb{G}$. This is achieved such that any $t + 1$ out of n parties can collectively reconstruct the secret key x, while any up to t of them remain oblivious to any information regarding the secret key x or the shares of honest parties. In [16], Pedersen proposed his well-known robust DKG protocol for a DL instance. In [12, 13], Gennaro, Jarecki, Krawczyk, and Rabin (GJKR) demonstrated a vulnerability in Pedersen DKG protocol [16], wherein an adversary could bias the distribution of the generated public key y. Subsequently, they used both Pedersen [17] and Feldman [11] VSS schemes and proposed a fully secure version of the Pedersen robust DKG protocol, mitigating the (computationally unbounded) adversary's ability to influence the distribution of the final public key y. It's worth to mention that the robustness property of DKG protocol guarantees the correct output for honest parties, even in the presence of a specific number of malicious parties. More recently, Atapoor, Baghery, Cozzo and Pedersen [1, Sec. 4.1] leveraged their new hash-based VSS scheme to introduce

a new variant of the Pedersen robust DKG protocol proposed by GJKR [12,13]. This variant [1] substantially reduces the computational cost for the parties by almost a factor of n. In their proposed DKG protocol, they use a commit-and-open approach to prevent possible attacks by a rushing adversary. However, in this work, we show that their protocol still is not fully-secure and a rushing adversary can bias the distribution of the generated public key. In an elegant recent work [15], Katz studied the round complexity of fully secure DKG protocols in the DL setting and presented several round-optimal (i.e., 3-round) DKG protocols, each offering different trade-offs in terms of security, efficiency, and setup requirements. Cascudo and David [9], as well as Kate et al. [14], also proposed 3-round, fully secure DKG protocols for the DL setting based on class groups, achieving one offline round (for public-key registration) followed by two online rounds. The protocol of Cascudo and David [9] has $O(n^2)$ computational complexity and is claimed to be concretely more efficient than the protocols introduced by Kate et al. in [15]. More recently, Boneh, Haitner, Lindell, and Segev [7] introduced an exponent verifiable random function (eVRF) and used it, together with Feldman VSS, to construct a DKG protocol in the DL setting. Similar to Katz's construction [15], their scheme requires two rounds of preprocessing and one online round; however, due to the use of Feldman VSS, parties must perform $O(n^2)$ exponentiations during the verification phase.

Our Contributions. The main contribution of this work is the presentation of several fully secure DKG protocols for the DL setting in the synchronous model, each achieving $O(n)$ computational complexity while offering different trade-offs among security, efficiency, and round complexity. Our key contributions can be summarized as follows:

A Unified Framework for DL-Oriented Pre-Constructed VSS. As a foundation for our new DKG protocols, we present a novel variant of the general VSS protocol Π [3], which we call PCΠ (Pre-Constructed Π). In contrast to the original framework, PCΠ requires the dealer to additionally publish a commitment to the main secret and leverages a new reconstruction method, termed *optimistic reconstruction*, adapted from the definition of pre-constructed publicly verifiable secret sharing [5]. We consider two concrete instantiations of PCΠ, and by combining their pre-constructability with the optimistic reconstruction technique, we design several more efficient DKG protocols with different trade-offs in terms of security, efficiency, and round complexity. We believe that PCΠ can serve as a more practical building block for the construction of distributed protocols than the original Π, and that the underlying ideas are general enough to extend naturally to other domains such as lattice- and isogeny-based cryptography. To construct PCΠ we propose an efficient Non-Interactive Threshold Zero-Knowledge (NI-TZK) proof scheme for the following distributed relations,

$$R_i = (y, x_i, f(X)) | y = g^{f(0)} \wedge f(i) = x_i, \quad i = 1, \ldots, n$$

where $f(X) \in \mathbb{Z}_q[X]_t$ is a secret polynomial in X of degree (at most) t with coefficients defined over the field $\mathbb{Z}_q$. This protocol is a NI-TZK proof scheme [1,6]

which enables a prover to convince n verifiers that the public key y has been correctly generated from the secret key $f(0)$, and that each verifier has received a valid Shamir share of this secret, i.e., $x_i = f(i)$. We believe that this protocol can serve as a useful building block for constructing a broader class of distributed protocols based on VSS schemes derived from the Π or PCΠ frameworks.

Statistically Fully Secure DKG Protocol for DL. In our first proposed DKG protocol, DKG_1, we combine the perfectly simulatable VSS scheme Π_{P} from [3] with a new hash-based PC-VSS scheme which is build using our proposed PCΠ framework. To construct DKG_1, we revisit the GJKR variant [12,13] of the Pedersen DKG protocol and propose a more efficient alternative that preserves full security while improving overall practicality. Similar to the original GJKR scheme, our protocol DKG_1 ensures that the final public key is uniformly random, even in the presence of up to t computationally unbounded adversaries, where t denotes the threshold parameter in Shamir's secret sharing.

Computationally Fully Secure DKG Protocol for DL. As another main contribution of this paper, we leverage the hash-based VSS scheme Π_{LA} proposed by Baghery [3] together with our new PC-VSS scheme $\text{PC}\Pi_{\text{F}}$ to revisit the GJKR DKG protocol and design a practical, fully secure DKG protocol for DL settings, denoted by DKG_2. This construction yields the first hash-based *fully secure* DKG protocol for DL, offering significantly higher efficiency than the GJKR protocol and, by a constant factor, better performance than our first DKG protocol DKG_1. While DKG_2 achieves full robustness and security against

Table 1. Asymptotic compuational and communication costs per-party in the fully-secure GJKR [12, 13] variant of Pedersen DKG [17], biased ABCP DKG [1], and our proposed DKG protocols. DL: Discrete Logarithm, RO: Random Oracle, Assu.: Assumption, rnd: Rounds, PP: pre-processing, BC: Broadcast, n: Number of parties, $t \approx n/2$: Threshold value, $E_{\mathbb{G}}$: Exponentiation in group $\mathbb{G}$, $\mathcal{PE}$: degree-t Polynomial Evaluation, $\mathcal{H}$: Hashing, $|\mathbb{G}|$: $\mathbb{G}$ element size, $|\mathbb{Z}_q|$: $\mathbb{Z}_q$ element size, $|\mathcal{H}|$: $\mathcal{H}$ image size.

DKG Scheme	Security, Round	Computation	Communication
GJKR [13] (Pedersen [17])	DL, fully-secure perfectly uniform, 6 rnd	$(2nt+n)\ E_{\mathbb{G}} + 2n\ \mathcal{PE} + 1\ \mathcal{H}$	PV: $2n\ \lvert\mathbb{Z}_q\rvert$, BC: $2t \approx n\ \lvert\mathbb{G}\rvert$
This Work Sec. 4	DL, RO, fully-secure perfectly uniform, 6 rnd	$8n\ E_{\mathbb{G}} + 5n\ \mathcal{PE} + 3n\ \mathcal{H}$	PV: $2n\ \lvert\mathbb{Z}_q\rvert$, BC: $n\ \lvert\mathbb{G}\rvert + n\ \lvert\mathcal{H}\rvert + 2t\ \lvert\mathbb{Z}_q\rvert$
ABCP [1]	DL, RO, biased, 4 rnd	$3n\ E_{\mathbb{G}} + 3n\ \mathcal{PE} + 5n\ \mathcal{H}$	PV: $n\ \lvert\mathbb{Z}_q\rvert$, BC: $2n\ \lvert\mathcal{H}\rvert + t\ \lvert\mathbb{Z}_q\rvert$
This Work Sec. 5	DL, RO, fully-secure compt. uniform, 6 rnd	$4n\ E_{\mathbb{G}} + 5n\ \mathcal{PE} + 6n\ \mathcal{H}$	PV: $n\ \lvert\mathbb{Z}_q\rvert$, BC: $n\ \lvert\mathcal{H}\rvert + 2t\ \lvert\mathbb{Z}_q\rvert + n\ \lvert\mathbb{G}\rvert$
Katz [15]	DL, RO, fully-secure comp. uniform, 3 rnd (2 PP)	$O(\binom{n}{t})\ E_{\mathbb{G}}$	PV: — BC: $O(n)$
Cas-Dav [9], Kate et al. [14]	DL, RO, fully-secure comp. uniform, 3 rnd (1 PP)	$O(n^2)\ E_{\mathbb{G}}$	PV: — BC: $O(n)$
BHLS [7]	DL, RO, fully-secure comp. uniform, 3 rnd (2 PP)	$O(n^2)\ E_{\mathbb{G}}$	PV: $O(n)$ BC: $O(n)$
This Work Sec. 6	DL, RO, fully-secure comp. uniform, 3 rnd	$O(n)\ E_{\mathbb{G}}$	PV: $O(n)$ BC: $O(n)$

up to t malicious parties, it guarantees that the distribution of the generated public key is only computationally (rather than perfectly) indistinguishable from uniform. Overall, DKG_2 can be viewed as a *fully secure* counterpart of the state-of-the-art *biased* DKG protocol proposed by ABCP [1].

Low-Round Computationally Fully Secure DKG Protocol for DL. As another key contribution of this paper, we introduce DKG_3 —a round-reduced variant of our fully secure and robust DKG protocol DKG_2. Compared to DKG_2, the new protocol requires only three rounds in the general case (instead of six), making it a scalable, practical, fully secure, and low-round DKG protocol for DL-based primitives. Notably, in comparison with the most recent round-optimal fully secure DKG protocols of Katz [15], Cascudo and David [9], and Boneh, Haitner, Lindell, and Segev [7]–all of which achieve three rounds via one online round plus two preprocessing rounds (or vice versa)–our DKG_3 protocol requires only $O(n)$ exponentiations, as opposed to $O(\binom{n}{t})$ or $O(n^2)$, and operates without any preprocessing phase or CRS setup.

Table 1 summarizes a comparison of the asymptotic costs of our proposed protocols and the most relevant robust DKG protocols. Protocols with similar properties are grouped together in the table.

2 Preliminaries

Notation. We let λ denote a security parameter. A function is called *negligible in* X, written $\mathsf{negl}(X)$, if for any constant c, there exists some X_0, such that $f(X) < X^{-c}$ for $X > X_0$. A function that is negligible in the security parameter λ is simply called negligible. We use the assignment operator $\leftarrow$ to denote uniform sampling from a set Ξ, e.g. $x \leftarrow \Xi$. We write $\mathbb{Z}_N := \mathbb{Z}/N\mathbb{Z}$ and $\mathbb{Z}_N[X]_t$ for polynomials of degree t in the variable X and with coefficients in $\mathbb{Z}_N$. For $n \in \mathbb{N}$, we write $[n] = \{1, \ldots, n\}$. Finally all logarithms are in base 2.

2.1 Zero-Knowledge Proofs on Secret Shared Data

In this section, we recall the definition of Non-Interactive Threshold Zero-Knowledge (NI-TZK) proofs [1, 6], a primitive that has been employed in several recently proposed VSS schemes [1, 3, 8], including the unified framework Π [3]. We build on this notion and also employ NI-TZK proofs in our proposed VSS protocols.

In a Threshold ZK proof a single prover interacts with several verifiers $\{V_i\}_{i=1}^n$ over a network that includes secure point-to-point channels, and each verifier V_j holds a share $x^{(j)} \in \mathbb{F}^{l_j}$ of the main statement x. The single prover aims to convince multiple verifiers that the main input x is in some language $L \subseteq \mathbb{F}^l$. Next, we recall the formal definition for distributed inputs (or statements), languages, relations, and threshold ZK proofs that are given by Boneh et al. [6].

Definition 1 (Distributed Inputs, Languages, and Relations [6]). *Let n be a number of parties, $\mathbb{F}$ be a finite field, and $l, l_1, l_2, \cdots, l_n \in \mathbb{N}$ be length parameters, where $l = l_1 + l_2 + \cdots + l_n$. An n-distributed input over $\mathbb{F}$ (or just*

distributed input/statement) is a vector $x = x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)} \in \mathbb{F}^l$ *where* $x^{(i)} \in \mathbb{F}^{l_i}$*, and it refers to a piece (or share) of* x*. An* n*-distributed language* L *is a set of* n*-distributed inputs. A distributed NP relation with witness length* h *is a binary relation* $R(x, w)$ *where* x *is an* n*-distributed input and* $w \in \mathbb{F}^h$*. We assume that all* x *in* L *and* $(x, w) \in R$ *share the same length parameters. Finally, we let* $L_R = \{x : \exists w (x, w) \in R\}$*.*

Definition 2 (*n*-Verifier Interactive Proofs [6]). *An* n-Verifier Interactive Proof *scheme over* $\mathbb{F}$ *is an interactive protocol* $\Gamma = (P, V_1, V_2, \cdots, V_n)$ *involving a prover* P *and* n *verifiers* $\{V_i\}_{i=1}^n$*. The protocol proceeds as follows.*

- *The prover* P *holds an* n*-distributed input* $x = x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)} \in \mathbb{F}^l$*, a witness* $w \in \mathbb{F}^h$*, and each verifier* V_j *holds an input share* $x^{(j)}$*.*
- *Parties can communicate in synchronous rounds over secure point-to-point channels. While honest parties follow the protocol, malicious parties (i.e., adversary) can send arbitrary messages.*
- *At the end, each verifier outputs either* 1 *(accept) or* 0 *(reject) based on its view, where the view of* V_j *consists of its input piece* $x^{(j)}$*, random input* $r^{(j)}$*, and messages it received during the execution of* Γ*.*

$\Gamma(x, w)$ denotes running Γ on secret shared input x and witness w, and says that $\Gamma(x, w)$ accepts (respectively, rejects) if at the end all verifiers output 1 (resp., 0). $View_{\Gamma,T}(x, w)$ denotes the (joint distribution of) views of verifiers $\{V_j\}_{j\in T}$ in the execution of Γ on the distributed statement x and witness w. Let $R(x, w)$ be a k-distributed relation over finite field $\mathbb{F}$. We say that an n-verifier interactive proof scheme $\Gamma = (P, V_1, \cdots, V_n)$ is a distributed threshold ZK proof protocol for R with t-security against malicious prover *and* malicious verifiers, and with soundness error ϵ, if Γ satisfies the following properties [1, 6]:

Definition 3 (Completeness). *For every* n*-distributed input* $x = x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)} \in \mathbb{F}^l$*, and witness* $w \in \mathbb{F}^h$*, such that* $(x, w) \in R$*, the execution of* $\Gamma(x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)}, w)$ *accepts with probability 1.*

Definition 4 (Soundness Against Prover and t Verifiers.). *For every* $T \subseteq [n]$ *of size* $|T| \leq t$*, an* $\mathcal{A}$ *controlling the prover* P *and verifiers* $\{V_j\}_{j\in T}$*,* n*-distributed input* $x = x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)} \in \mathbb{F}^l$*, and witness* $w \in \mathbb{F}^h$ *the following holds. If there is no* n*-distributed input* $x' \in L_R$ *such that* $x'_H = x_H$*, where* $H = [n]/T$*, the execution of* $\Gamma^\star(x, w)$ *rejects except with at most* ϵ *probability, where here* $\Gamma^\star$ *denotes the interaction of* $\mathcal{A}$ *with the honest verifiers.*

Definition 5 (Threshold ZK). *For every* $T \subseteq [n]$ *of size* $|T| \leq t$ *and an* $\mathcal{A}$ *controlling* $\{V_j\}_{j\in T}$*, there exists a simulator* $\mathcal{S}$ *such that for every* n*-distributed input* $x = x^{(1)} \parallel x^{(2)} \parallel \cdots \parallel x^{(n)} \in \mathbb{F}^l$*, and witness* $w \in \mathbb{F}^h$ *such that* $(x, w) \in R$*, we have* $\mathcal{S}((x^{(j)})_{j\in T}) \equiv View_{\Gamma^\star,T}(x, w)$*. Here,* $\Gamma^\star$ *denotes the interaction of adversary* $\mathcal{A}$ *with the honest prover* P *and the honest verifiers* $\{V_j\}_{j\in T}$*.*

Remark 1 (Threshold Honest-Verifier ZK). In the context of Threshold ZK, one may consider a relaxed definition, Threshold *Honest-Verifier* ZK, that retains the same properties as the original definition, with the added requirement that the subset of verifiers, $\{V_j\}_{j\in T}$, is stipulated to follow the protocol honestly.

2.2 Verifiable Secret Sharing

Our studied protocols utilize Shamir secret sharing [19] for securely sharing a secret, a concept we review below.

Shamir Secret Sharing. A $(t+1, n)$-Shamir secret sharing scheme [19] allows n parties to individually hold a share x_i of a common secret f_0, such that any subset of t parties or less are not able to learn any information about the secret f_0, while any subset of at least $t+1$ parties are able to efficiently reconstruct the common secret f_0. In more detail, this is achieved via polynomial interpolation over the field $\mathbb{Z}_q$. A common polynomial $f(x) \in \mathbb{Z}_q[x]_t$ is chosen, such that the secret f_0 is set to be its constant term, namely $f_0 = f(0)$. Each party P_i for $i \in \{1, \cdots, n\}$ is assigned the secret share $f_i = f(i)$. Then any subset $Q \subseteq \{1, \ldots, n\}$ of at least $t+1$ parties can reconstruct the secret f_0 via Lagrange interpolation by computing $f_0 = f(0) = \sum_{i \in Q} f_i \cdot L_{0,i}^Q$, where

$$L_{0,i}^Q := \prod_{j \in Q \setminus \{i\}} \frac{j}{j-i} \pmod q.$$

are the Lagrange basis polynomials evaluated at 0. Any subset of up to t parties are not able to find $f_0 = f(0)$, as this is Information-Theoretically (IT) hidden from the other shares.

Verifiable Secret Sharing (VSS). A typical secret sharing scheme is designed to withstand passive attacks. However, in numerous scenarios, it's imperative for a secret sharing scheme to also be resilient against malicious actors or parties employing active attacks. This enhanced security is achieved through VSS schemes, first introduced in 1985 [10]. Next, we recall the definition of computational VSS schemes, adapted from [3,17], and give a brief overview of several constructions most relevant to this work. In these definitions, the dealer employs NI-TZK proofs [1,6] to demonstrate the correctness of the distributed shares, while verification requires the participation of at least $t+1$ honest parties.

Definition 6. *An (n, t, f_0)-VSS consists of four polynomial time algorithms of (Initial, Share, Verify, Reconstruct) as follows:*

1. $\mathsf{Initial}(1^\lambda) \to pp$*: Given 1^λ, the public parameters pp are sampled and shared with the parties.*
2. $\mathsf{Share}(n, t, f_0, pp) \to (\{f_i\}_{i=1}^n, \pi_{share})$*: Given secret f_0, it secret shares f_0 and obtains the shares $\{f_i\}_{i=1}^n$. Then, it outputs the shares $\{f_i\}_{i=1}^n$, and a (non-interactive) threshold proof π_{share}, to prove the validity of the shares.*
3. $\mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, \pi_{share}) \to$ `true/false`*: Given (n, t), the shares $\{f_i\}_{i=1}^n$, and the proof π_{share}, the algorithm outputs either* `true`*/*`false`*.*
4. $\mathsf{Reconstruct}(\{f_i\}_{i \in Q, |Q| \geq t+1}, \pi_{share}) \to \{f_0, \mathsf{false}\}$*: Given any at least $t+1$ of the shares and the proof π_{share}, it returns either the secret f_0, or* `false`*.*

A secure VSS scheme needs to satisfy the following properties.

- **Correctness**: For any integers $n > 1$ and $t < n$ a VSS is called correct, if

$$\Pr\begin{bmatrix} pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, \pi_{share}) \leftarrow \mathsf{Share}(n,t,f_0,pp) : \\ \mathtt{true} \leftarrow \mathsf{Verify}(n,t,\{f_i\}_{i=1}^n, \pi_{share}) \end{bmatrix} = 1,$$

$$\Pr\begin{bmatrix} pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, \pi_{share}) \leftarrow \mathsf{Share}(n,t,f_0,pp), \\ f_0' \leftarrow \mathsf{Reconstruct}(\{f_i\}_{i\in Q, |Q|\geq t+1}, \pi_{share}) : f_0' = f_0 \end{bmatrix} = 1 \ .$$

- **Verifiability**: A shareholder must be able to verify the validity of the received share and if the Verify algorithm returns true, then for any qualified set of honest parties can reconstruct a unique secret s. More formally, given λ, for any integers $n \geq 2t+1$ and $t \geq 0$, a VSS is called verifiable if for any PPT adversaries $\mathcal{A}$, we have:

$$\Pr\begin{bmatrix} pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, \pi_{share}) \leftarrow \mathcal{A}(n,t,pp), \\ \mathtt{true} \leftarrow \mathsf{Verify}(n,t,\{f_i\}_{i=1}^n, \pi_{share}) : \\ \exists s, \forall Q \in [n], s \leftarrow \mathsf{Reconstruct}(\{f_i\}_{i\in Q, |Q|\geq t+1}, \pi_{share}) \end{bmatrix} \geq 1 - \mathsf{negl}(\lambda) \ ,$$

  where Q is the set of qualified and honest parties.

- **Unpredictability**: For any integers $n > 1$ and $t < n$, a VSS is called unpredictable if for all PPT adversaries $\mathcal{A}$ who has non-adaptively corrupted up to t parties, for all λ, and a random secret f_0, we have:

$$\Pr\begin{bmatrix} pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, \pi_{share}) \leftarrow \mathsf{Share}(n,t,f_0,pp); \\ Q \subset [n]; |Q| \leq t; f_0' \leftarrow \mathcal{A}(n,t,\{f_i\}_{i\in Q}, \pi_{share}) : f_0' = f_0 \end{bmatrix} \leq \mathsf{negl}(\lambda) \ .$$

- **Simulatability:** In some cases, the definition of unpredictability can be amplified by requiring that the view of corrupted parties can be simulated. Intuitively, this means that even if an adversary corrupts up to t parties, its joint view can be reproduced without knowing the secret.

Feldman and Pedersen VSS Protocols. For brevity, we refer to App. A for a summary of the classic Feldman [11] and Pedersen [17] VSS schemes.

Perfectly Simulatable VSS Scheme Π_{P}. In [3], Baghery presented Π_{P} as an efficient alternative to the Pedersen VSS scheme that can achieve perfect simulatability. In Π_{P}, given (n,t,f_0) and random group generators $pp := (g_1, g_2, g_3)$, to share a secret f_0, the Share, Verify and Reconstruct algorithms act as below:

- $\mathsf{Share}(n,t,f_0,pp) \rightarrow (\{f_i\}_{i=1}^n, \pi_{share})$: Given $pp := (g_1,g_2,g_3)$, the parameters (n,t), random oracle $\mathcal{H}$, to share f_0, the dealer proceeds as follows: Sample two uniformly random polynomials $f(X)$ and $r(X)$ of degree t with coefficients in $\mathbb{Z}_q$, subject to $f(0) = f_0$. For $i = 1, 2, \cdots, n$: set $f_i := f(i)$, and $r_i := r(i)$. For $i = 1, 2, \cdots, n$: set $c_i = g_1^{f_i} g_2^{r_i} g_3^{\gamma_i}$, where γ_i are random elements from $\mathbb{Z}_q$. Compute the challenge value $d := \mathcal{H}(c_1, \ldots, c_n)$;

Check if $g_1 \neq g_2^d$ and if the check passed, set $z(X) = r(X) + d \cdot f(X)$ and $\pi_{share} := (c_1, \ldots, c_n, z(X))$. Otherwise, restart the protocol from third step; Send shares (f_i, γ_i) securely to party P_i and return π_{share}.

- $\mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, \pi_{share}) \rightarrow$ `true/false`: Given n, threshold value t, group generators $pp := (g_1, g_2, g_3)$, a proof $\pi_{share} := (c_1, \ldots, c_n, z(X))$, and the shares $\{f_i, \gamma_i\}_{i=1}^n$: Each party P_i first checks if $z(X)$ is a degree t polynomial, computes challenge value $d = \mathcal{H}(c_1, \ldots, c_n)$ and then uses his/her shares (f_i, γ_i) and checks if $c_i = g_1^{f_i} g_2^{z(i) - d \cdot f_i} g_3^{\gamma_i}$. If the verification of P_i fails, then P_i broadcasts a complain against the dealer. Any possible conflict between the dealer and the parties will be solved using a known conflict resolution approach (used in Feldman/Pedersen VSS schemes).
- $\mathsf{Reconstruct}(\{f_i\}_{i \in Q, |Q| = t+1}) \rightarrow \{f_0, \mathsf{false}\}$: Each party P_i broadcasts the secret values f_i and γ_i. A party P_i is said to be confirmed if $c_i = g_1^{f_i} g_2^{z(i) - d \cdot f_i} g_3^{\gamma_i}$, where $d := \mathcal{H}(c_1, \ldots, c_n)$. Consider f_i values of any $t+1$ confirmed parties and interpolate $f(X)$ of degree t that pass through those points. Finally, the output is $f_0 = f(0)$ or `false` (if $t+1$ valid shares were not obtained).

In [3], Baghery also presented Π_{F} VSS scheme, which is an efficient alternative to Feldman VSS scheme. Different from Π_{P}, in Π_{F} commitments are computed as $c_i = g_1^{f_i} g_2^{r_i}$ for $i = 1, \ldots, n$. More recently, Atapoor et al. [2] proposed a slightly more efficient variant of the Π_{P} VSS scheme, referred to as $\Pi_{\mathrm{P}}+$, which achieves a constant-factor performance improvement. In contrast to Π_{P}, $\Pi_{\mathrm{P}}+$ computes commitments as $c_i = g_1^{\mathsf{H}(f_i, r_i)} g_2^{\gamma_i}$ for $i = 1, \ldots, n$, where H denotes a suitably defined secure hash function.

Computationally Simulatable Hash-Based VSS Scheme Π_{LA}. In [3], Baghery presented Π_{LA}, which can be seen as the optimized version of Atapoor, Baghery, Cozzo, and Pedersen VSS scheme [1]. In Π_{LA}, given a secure hash-based commitment H, random oracle $\mathcal{H}$, the parameters n and t, to share a high-entropy secret f_0, the Share, Verify and $\mathsf{Reconstruct}$ algorithms act as follows:

- $\mathsf{Share}(n, t, f_0, pp) \rightarrow (\{f_i\}_{i=1}^n, \pi_{share})$: Given the hash-based commitment scheme H, the parameters (n, t), random oracle $\mathcal{H}$, to share f_0, the dealer proceeds as follows: Sample a uniformly random $f(X)$ and $r(X)$ of degree t with coefficients in $\mathbb{Z}_q$, subject to $f(0) = f_0$. For $i = 1, \cdots, n$: set $f_i := f(i)$, $r_i := r(i)$, and set $c_i = \mathsf{H}(f_i, r_i)$. Compute the challenge value $d := \mathcal{H}(c_1, \ldots, c_n)$; Set $z(X) = r(X) + d \cdot f(X)$ and $\pi_{share} := (c_1, \ldots, c_n, z(X))$. Send share f_i securely to party P_i and return π_{share}.
- $\mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, \pi_{share}) \rightarrow$ `true/false`: Given (n, t), the hash-based commitment scheme H, a proof $\pi_{share} := (c_1, \ldots, c_n, z(X))$, and the shares $\{f_i\}_{i=1}^n$: Each party P_i first checks if $z(X)$ is a degree t polynomial, computes challenge value $d = \mathcal{H}(c_1, \ldots, c_n)$ and then uses his/her shares f_i and checks if $c_i = \mathsf{H}(f_i, z(i) - d \cdot f_i)$. If the verification of P_i fails, then P_i broadcasts a complain against the dealer. Any possible conflict between the dealer and the parties will be solved using a known conflict resolution approach.

- $\mathsf{Reconstruction}(\{f_i\}_{i\in Q,|Q|=t+1}) \rightarrow \{f_0, \mathsf{false}\}$: Each party P_i broadcasts the secret values f_i. A party P_i is said to be confirmed if $c_i = \mathsf{H}(f_i, z(i) - d \cdot f_i)$, where $d := \mathcal{H}(c_1, \ldots, c_n)$. Consider f_i values of any $t+1$ confirmed parties and interpolate $f(X)$ of degree t that pass through those points. Finally, the output is $f_0 = f(0)$ or `false` (if $t+1$ valid shares were not obtained).

Round Optimal Variant of Π VSS Scheme. Recently, Baghery, Bardeh, Khazaei, and Rahimi [4] introduced a round-optimal (i.e., 2-round) variant of the unified Π framework [3], referred to as 2R-Π. This framework enables the construction of practical round-optimal VSS schemes in the honest-majority setting. Building on this, they proposed a round-optimal version of all the VSS schemes built by Π, namely (Π_{P}, Π_{F}, Π_{LA}), presented in [3]. In 2R-Π_{F}, given a the group generators (g_1, g_2), a random oracle $\mathcal{H}$, the parameters n and t, and a high-entropy secret f_0, the dealer and parties proceed as outlined in Fig. 1.

2.3 Distributed Key Generation

A robust Distributed Key Generation (DKG) protocol for DL is an interactive cryptographic protocol executed by n parties $P_1, \ldots, P_n$. Given a security parameter λ, a fixed generator g of a cyclic group $\mathbb{G}$, and a threshold parameter t, the protocol outputs a public key $y = g^x$ together with individual shares $(x_1, \ldots, x_n)$ of the hidden secret key $x \in \mathbb{Z}_q$. Some DKG protocols in the DL setting further ensure that parties can compute public commitments to each others' shares, as such commitments are often required in threshold protocols.

Robust DKG protocols generally proceed in two phases, with VSS schemes playing a central role in their design. In the first phase, each party performs *verifiable secret sharing* of its secret input and commits to it, preventing a rushing adversary from biasing the distribution of the final public key. A fully secure robust DKG protocol must in particular guarantee that the adversary cannot influence or bias the distribution of the generated public key y. In the second phase, parties combine the outcomes of the first phase to compute the final public key. The *robustness* property ensures that the protocol always produces correct outputs, even when up to t malicious parties attempt to disrupt execution or behave dishonestly.

A robust DKG is called t-secure (or secure with threshold t) if in the presence of an attacker corrupting at most t parties, it satisfies the following requirements for *correctness* and *secrecy* [12, 13]:

- **Correctness**:

(C1) There is an efficient procedure that on input the n shares submitted by the parties and the public information produced by the DKG protocol, outputs the unique value x, even if up to t shares are submitted by malicious parties. The latter shows that the procedure should be robust.

(C2) All honest parties have the same value of public key $y = g^x$, where x is the unique secret guaranteed by (C1).

Sharing: Two Rounds

Round 1: Given the group generators $pp := (g_1, g_2)$, a secret f_0, dealer D

1. Chooses a random polynomial $f(X)$ and $r(X)$ of degree-t such that $f(0) = f_0$
2. For $i = 1, 2, ..., n$: set $f_i := f(i)$ and $r_i := r(i)$.
3. For $i = 1, 2, ..., n$: set $c_i = g_1^{f_i} g_2^{r_i}$.
4. Compute the challenge value $d = \mathcal{H}(c_1, c_2, \ldots, c_n)$, where $\mathcal{H} : \{0,1\}^* \to \mathbb{Z}_q$ is an instantiation for the Random Oracle.
5. Set $z(X) = r(X) + df(X)$ and $\pi_{share} := (c_1, \ldots, c_n, z(X))$.
6. Send share f_i securely to party P_i and broadcast π_{share}.

Given the group generator g_1, every other party P_i

1. Chooses a random value s_i and compute $c_{s_i} = g_1^{s_i}$. Note that, in this case we can omit the randomizer t_i mentioned in the general construction.
2. Send s_i to dealer, and broadcast commitment c_{s_i}.

Round 2: Dealer D, for every party P_i

1. Using the algorithm Open of the commitment scheme, checks if commitment c_{s_i} is consistent with the received value s_i.
2. Broadcast $\alpha_i = f_i + s_i$ if the verification succeeds, and broadcast f_i otherwise.

Party P_i

1. First verify if $\deg(z(X)) = t$ and, then using his/her share f_i checks if $c_i = g_1^{f_i} g_2^{z(i) - df_i}$, where $d = \mathcal{H}(c_1, c_2, \ldots, c_n)$.
2. Broadcasts nothing if the verification succeeds, and broadcasts s_i otherwise. Party P_i is considered happy in the former case, while he/she is unhappy in the later case.

Local Computation: Every party P_k

1. discards D and halts the execution of the protocol, if
   (a) D broadcasts f_i such that $c_i \neq g_1^{f_i} g_2^{z(i) - df_i}$, where $d = \mathcal{H}(c_1, c_2, \ldots, c_n)$.
   (b) D broadcasts α_i; and P_i broadcasts s_i such that $c_{s_i} = g_1^{s_i}$ and $c_i \neq g_1^{f_i'} g_2^{z(i) - df_i'}$, where $d = \mathcal{H}(c_1, c_2, \ldots, c_n)$ and $f_i' = \alpha_i - s_i$.
2. Discards an unhappy party P_i, if he/she broadcasts (s_i, t_i) such that $c_{s_i} \neq g_1^{s_i} g_2^{t_i}$. Let $\mathbb{Q}$ be the set of non-discarded parties.
3. Outputs f_k as received in Round 1, if P_k is happy and in $\mathbb{Q}$. If he/she is unhappy and belongs to $\mathbb{Q}$, then he/she outputs f_k if they are directly broadcastebd by D in Round 2. Otherwise, P_k computes f_k as $f_k = \alpha_k - s_k$.

Reconstruction: One Round

1. Party $P_i \in \mathbb{Q}$ broadcast f_i'.

Local Computation: For every party P_k,

1. Party $P_i \in \mathbb{Q}$ is said to be *confirmed* if $c_i = g_1^{f_i'} g_2^{z(i) - df_i'}$ where $d = \mathcal{H}(c_1, c_2, \ldots, c_n)$.
2. Consider f_i' values of any $t+1$ *confirmed* parties and interpolate $f'(X)$. Output $f_0' = f'(0)$.

Fig. 1. 2-Round Π_{F}: a round-optimal variant of Π_{F} in the honest majority setting [4].

(C3) The secret key x is uniformly distributed in $\mathbb{Z}_q$ (and hence the public key y is uniformly distributed in the subgroup generated by g).

- **Secrecy**: No information on secret key x can be learned by the adversary except for what is implied by the value $y = g^x$. More formally, in terms of simulatability, for every PPT adversary $\mathcal{A}$ controlling up to t parties, there exists a PPT simulator $\mathcal{S}$, such that on input an element $y \in \mathbb{G}$, produces an output distribution which is polynomially indistinguishable from $\mathcal{A}$'s view of a run of the DKG protocol that ends with y as its public key output.

Remark 2 (Relaxed Requirements for DKG.). In practical implementations, such as in our protocols, the aforementioned conditions (C1) and (C2) may be relaxed to permit a negligible probability of error. Similarly, condition (C3) can be relaxed to accommodate a distribution for x that exhibits negligible statistical deviation from the uniform distribution.

2.4 A Σ-Protocol for Pedersen Commitment and DL Equality

Let $\mathbb{G}$ be a group with hard DL, and (g_1, g_2, g_3) be three random group generators of it. Let a prover aim to convince a verifier that for the public statement g_1, g_2, g_3, a, b, he knows a witness (x, y) which holds in the following relation,

$$R_{Ped-DLEQ} = \{(g_1, g_2, g_3, a, b), x, y \mid a = g_1^x g_2^y \wedge b = g_3^x\}. \tag{1}$$

One can construct a secure and efficient Σ-protocol for the above relation by taking the conjunction of a Σ-protocol proving knowledge of the message of a Pedersen commitment and a Σ-protocol proving knowledge of a discrete logarithm witness. The resulting Σ-protocol can be transformed into a non-interactive zero-knowledge argument of knowledge in the random oracle model via the Fiat-Shamir transform. We implement this transformation and summarize the resulting protocol in Fig. 2. This protocol is used later as a building block in the construction of one of our proposed DKG protocols.

Prover: Given the statement $(g_1, g_2, g_3, a, b) \in \mathbb{G}$ and the witness value $(x, y) \in \mathbb{Z}_q$, proceed as follows and output a proof π.

1. Sample $r_1, r_2 \leftarrow\$ \mathbb{Z}_q$ uniformly at random; and set $c_1 = g_1^{r_1} g_2^{r_2}$ and $c_2 = g_3^{r_1}$.
2. Set $d \leftarrow \mathcal{H}(a, b, c_1, c_2)$, where $\mathcal{H}$ is a random oracle.
3. Set $z_1 = r_1 + d \cdot x \mod q$ and $z_2 = r_2 + d \cdot y \mod q$; and Return $\pi := (d, z_1, z_2)$

Verifier: Given the statement $(g_1, g_2, g_3, a, b) \in \mathbb{G}$ and the proof $\pi = (d, z_1, z_2)$, checks if $d = \mathcal{H}(a, b, \frac{g_1^{z_1} g_2^{z_2}}{a^d}, \frac{g_3^{z_1}}{b^d})$ and outputs `true` or `false`.

Fig. 2. An efficient NIZK argument of knowledge for relation (1).

3 Pre-Constructed Variant of Π Protocol

In this section, we introduce a new variant of the general VSS protocol Π, called PCΠ (Pre-Constructed Π). Unlike the original Π protocol, PCΠ requires the dealer to additionally publish a commitment to the main secret and incorporates a novel reconstruction technique, introduced for pre-constructed publicly verifiable secret sharing [5], known as *optimistic reconstruction*. As we will show later, this reconstruction approach enables the design of more efficient distributed protocols. The PCΠ protocol and its instantiations play a central role in our constructions of presented DKG protocols.

3.1 Definitions of Pre-Constructed VSS Schemes

We begin by formalizing the syntax for Pre-Constructed VSS (PC-VSS) schemes, adapted from [5] with some simplifications. This syntax can also be seen as a natural extension of the VSS syntax [1, 3, 17] to support pre-constructability.

Definition 7. *A Pre-Constructed VSS (PC-VSS) protocol consists of four algorithms of (Initial, Share, Verify, Reconstruct) as follows:*

1. $\mathsf{Initial}(1^\lambda) \to pp$*: Given* 1^λ*, the public parameters pp are sampled and shared with the parties. In this case, pp also includes a commitment key* h_0 *which will be used in the sharing phase while committing to the main secret.*
2. $\mathsf{Share}(n, t, f_0, pp) \to (\{f_i\}_{i=1}^n, c_0, \pi_{share})$*: Given secret* f_0*, it secret shares* f_0 *and obtains the shares* $\{f_i\}_{i=1}^n$*. Next, it commits to* f_0 *and obtains the commitment* c_0*. It finally outputs the shares* $\{f_i\}_{i=1}^n$*, the commitment* c_0*, and a threshold proof* π_{share}*, to prove the validity of the shares and* c_0*.*
3. $\mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, c_0, \pi_{share}) \to$ `true/false`*: Given* (n, t)*, the shares* $\{f_i\}_{i=1}^n$*, the commitment* c_0*, and the proof* π_{share} *generated by* Share*, the algorithm outputs either* `true`*/*`false`*.*
4. Reconstruct*: This phase can be done in two ways:*
   (a) *(Optimistic)* $\mathsf{Reconstruct}^{opt}(pp, c_0, f_0, r_0) \to \{f_0, \mathsf{false}\}$*: Given public parameters pp, the commitment value* c_0 *and the opening value* f_0 *(or* (f_0, r_0) *if applicable), it checks if* f_0 *is a valid opening for* c_0*. If so, it returns* f_0*; otherwise it returns* `false`*.*
   (b) *(Pessimistic)* $\mathsf{Reconstruct}^{pes}(\{f_i\}_{i\in Q, |Q|\geq t+1}, \pi_{share}) \to \{f_0, \mathsf{false}\}$*: Given any at least* $t+1$ *of the shares and the proof* π_{share}*, it returns either the secret* f_0*, or* `false`*.*

A secure PC-VSS needs to satisfy the following security guarantees.

- **Correctness**: If the dealer and parties follow the protocol, then the Verify algorithm will return `true` and the Reconstruct by both approaches will return f_0. More formally, for any integers $n > 1$ and $t < n$ a PC-VSS is called correct, if for $pp \leftarrow \mathsf{Initial}(1^\lambda)$, we have:

$$\Pr\begin{bmatrix}(\{f_i\}_{i=1}^n, c_0, \pi_{share}) \leftarrow \mathsf{Share}(n, t, f_0, pp) : \\ \mathtt{true} \leftarrow \mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, c_0, \pi_{share})\end{bmatrix} = 1,$$

$$\Pr\begin{bmatrix}(\{f_i\}_{i=1}^n, c_0, \pi_{share}) \leftarrow \mathsf{Share}(n, t, f_0, pp), \\ f_0' \leftarrow \mathsf{Reconstruct}^{opt}(pp, c_0, f_0, r_0) \quad \vee \\ f_0' \leftarrow \mathsf{Reconstruct}^{pes}(\{f_i\}_{i\in Q, |Q|\geq t+1}, \pi_{share}) : f_0' = f_0\end{bmatrix} = 1 \ .$$

- **Verifiability**: A shareholder must be able to verify the validity of the received share and the commitment to the main secret. If the Verify algorithm returns true, then for any qualified set of honest parties can reconstruct a unique secret f_0 using both the reconstruction approaches $\mathsf{Reconstruct}^{opt}$ and $\mathsf{Reconstruct}^{pes}$. More formally, given λ, for any integers $n \geq 2t+1$ and $t \geq 0$, a PC-VSS is called verifiable if for any PPT adversaries $\mathcal{A}$, we have:

$$\Pr\begin{bmatrix}pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, c_0, \pi_{share}) \leftarrow \mathcal{A}(n, t, pp), \\ \mathtt{true} \leftarrow \mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, c_0, \pi_{share}) : \\ \exists s, \forall Q \in [n], s \leftarrow \mathsf{Reconstruct}^{pes}(\{f_i\}_{i\in Q, |Q|\geq t+1}, \pi_{share}), \\ \vee \ \ s \leftarrow \mathsf{Reconstruct}^{opt}(pp, c_0, s, r_0)\end{bmatrix} \geq 1 - \mathsf{negl}(\lambda) \ ,$$

  where Q is the set of honest parties.

- **Unpredictability**: The protocol must be unpredictable, meaning that there is no strategy for selecting t shares of the secret that would enable someone to predict the secret f_0 with a significant advantage. More formally, for any integers $n > 1$ and $t < n$, a PC-VSS is called unpredictable if for all PPT adversaries $\mathcal{A}$ who has non-adaptively corrupted up to t parties, for all λ, and a random secret f_0, we have:

$$\Pr\begin{bmatrix} pp \leftarrow \mathsf{Initial}(1^\lambda), (\{f_i\}_{i=1}^n, c_0, \pi_{share}) \leftarrow \mathsf{Share}(n, t, f_0, pp); \\ Q \subset [n]; |Q| \leq t; f_0' \leftarrow \mathcal{A}(n, t, \{f_i\}_{i\in Q}, c_0, \pi_{share}) : f_0' = f_0 \end{bmatrix} \leq \mathsf{negl}(\lambda) \ .$$

- **Simulatability:** In some cases, we can amplify the unpredictability guarantee by requiring that the view of corrupted parties can be simulated. More formally, a PC-VSS protocol Π satisfies *simulatability* if for every PPT adversary $\mathcal{A}$ corrupting an unqualified set U with $|U| \leq t$, for every secret $s \in \mathbb{Z}_q$, for every security parameter λ, and for $pp \leftarrow \mathsf{Initial}(1^\lambda)$, there exists a simulator $\mathcal{S}$ such that:

$$\begin{aligned} &\text{Let } \ (\{f_i\}_{i=1}^n, c_0, \pi_{share}) \leftarrow \mathsf{Share}(n, t, s, pp), \\ &\text{Then } \mathcal{S}\big(U, c_0, \{f_j\}_{j\in U}\big) \ \approx \ \text{View}_{\Pi,\mathcal{A}}\big(\{f_i\}_{i=1}^n, c_0, \pi\big). \end{aligned}$$

  This ensures that an adversary $\mathcal{A}$ corrupting at most t parties learns nothing beyond its legitimate shares and the public commitment c_0. We remark in practice, this definition can be slightly weaker than the notion of simulatability achieved in VSS schemes like Pedersen [17] and Π_{P} [3]. The reason is that in this setting the adversary may still obtain a computational ZK proof π_{share} or a computationally hiding function of the secret, e.g., g^s as in the case of DKG protocols, where g is the group generator.

3.2 A DL Oriented NI-TZK Protocol for PCΠ Framework

For our protocols, we require a tailored NI-TZK proof scheme, which we construct in this section and refer to as the *DL-oriented NI-TZK protocol for* PCΠ. Given a group generator g, this protocol enables a prover (e.g., a dealer) to convince n verifiers (or shareholders) that the published DL commitment $y = g^{f(0)}$ is correctly computed from $f(0)$, and that each verifier $i \in [n]$ has received a valid evaluation $x_i := f(i)$ of a polynomial $f(X)$ of degree at most t –equivalently, a valid Shamir share of $f(0)$. Formally, the DL-oriented NI-TZK protocol for PCΠ is defined over the following n-distributed relations:

$$R_i = \{(y, x_i, f(X)) \mid y = g^{f(0)} \wedge x_i = f(i)\}, \quad i \in [n], \quad (n \geq 2t+1), \qquad (2)$$

where $f(X) \in \mathbb{Z}_q[X]_t$ is a secret polynomial of degree at most t with coefficients in $\mathbb{Z}_q$. The algorithms of the proposed DL-oriented NI-TZK protocol for these relations are depicted in Fig. 3. Intuitively, the construction of the DL-oriented NI-TZK scheme (Fig. 3) can be seen as the conjunction of two components: (i) the NI-TZK protocol used in the general Π VSS protocol [3], and (ii) the classic

Prover: Given, $f(X) \in \mathbb{Z}_q[X]_t$, and the input $x = (g, y := g^{f(0)}, x_1, \cdots, x_n)$, proceed as follows and output a proof π of the distributed relations in Eq. (2).

1. Sample $r(X) \leftarrow \mathbb{Z}_q[X]_t$ uniformly at random;
2. For $i = 1, \ldots, n$: set $\mathsf{C}_i = \mathcal{C}(x_i, r(i))$, where $\mathcal{C}$ is a commitment scheme;
3. Set $\mathsf{C}_0 = g^{r(0)}$ and $d \leftarrow \mathcal{H}(g, y, \mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n)$, where $\mathcal{H}$ is a random oracle;
4. Set $z(X) \leftarrow r(X) - d \cdot f(X) \mod q$;
5. Return $\pi := (g, y, \mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n, z(X))$;

Verification: Given $\pi := (g, y, \mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n, z(X))$, the verifiers $\{V_i\}_{i=1}^n$ use their individual secret shares x_i and engage in the verification process as follows:

1. Verifier i acts as below and outputs `true` or `false`.
   (a) Set $d \leftarrow \mathcal{H}(g, y, \mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n)$;
   (b) If $\deg(z(X)) \leq t$, and $\mathsf{C}_i == \mathcal{C}(x_i, z(i) + d \cdot x_i) \wedge \mathsf{C}_0 == g^{z(0)} y^d$, return `true`; otherwise `false`;
2. Return `true` if *all* the verifiers return `true`; otherwise returns `false`.

Fig. 3. The DL oriented NI-TZK proof for the n-distributed relation defined in Eq. (2).

Schnorr identification protocol [18]. A key property of the proposed scheme is that verification can be carried out using any $t+1$ honest evaluations $x_i = f(i)$, privately held by the verifiers (or shareholders). The security of this protocol is established in the following theorem.

Theorem 1 (DL Oriented NI-TZK Protocol for PCΠ). *Let L be an n-distributed language for the list of relations given in Eq. (2), $t \geq 1$ be a security threshold such that $n \geq 2t+1$. Assuming that $\mathcal{C}$ is a computationally hiding hash-based (or DL-based) commitment scheme and the DL problem is computationally hard, for any potential set $I \subseteq [n]$ of size $|I| \geq n - t$, the protocol described in Fig. 3 is a NI-TZK for L that satisfies completeness, threshold ZK, and (special) soundness against the prover and t malicious verifiers in the RO model.*

Proof. The completeness is straightforward, as for $i = 1, \ldots, n$,

$$\mathcal{C}(x_i, z(i) + d \cdot x_i) = \mathcal{C}(x_i, r(i) - dx_i + dx_i) = \mathcal{C}(x_i, r(i)) = \mathsf{C}_i, \quad \text{and}$$

$$g^{z(0)} y^d = g^{r(0) - df(0)} g^{df(0)} = g^{r(0)} = \mathsf{C}_0 \ .$$

For the (special) soundness against the prover and t malicious verifiers, we show that given two acceptable transcripts of the protocol, we can build an efficient extraction algorithm that can extract a witness from the prover. Let,

$$(\mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n, d, z(X)) \quad \text{and} \quad (\mathsf{C}_0, \mathsf{C}_1, \ldots, \mathsf{C}_n, d', z'(X))$$

be two acceptable transcripts of the protocol, obtained by rewinding the prover. Then, from the verification equation we know that, for $i = 1, \ldots, n$,

$$\mathsf{C}_i == \mathcal{C}(x_i, z(i) + d \cdot x_i) \wedge \mathsf{C}_0 == g^{z(0)} y^d \quad \text{and}$$

$$\mathsf{C}_i == \mathcal{C}(x_i, z'(i) + d' \cdot x_i) \wedge \mathsf{C}_0 == g^{z'(0)} y^{d'} \ .$$

Using the last two equations and relying on the binding property of the commitment scheme $\mathcal{C}$, we conclude that, for $i = 1, \ldots, n$,

$$z(i) + dx_i = z'(i) + d'x_i \quad \text{and} \quad g^{z(0)}y^d = g^{z'(0)}y^{d'} \ ,$$

$$\Rightarrow \quad x_i = \frac{z'(i) - z(i)}{d - d'} \quad \text{and} \quad \log_g y = \frac{z'(0) - z(0)}{d - d'} \ .$$

Taking into account that $n \geq 2t+1$ and $z(X)$ represents a polynomial of degree t, if the verification algorithm returns `true`, then it enables an extractor to use the shares of any $t+1$ honest parties and Lagrange interpolation and reconstruct a unique degree-t witness polynomial $f(X)$ from the provided equations. Conversely, from the right hand equation, an extractor can extract $f(0)$, which serves as a witness for the DL problem.

For the threshold ZK proof, it is essential to demonstrate the *simulatability* of the view of any adversary $\mathcal{A}$ that corrupts up to t verifiers. We analyze the two commitment schemes $\mathcal{C}$ considered in this paper and describe how the simulator $\mathcal{S}$ can generate an indistinguishable transcript [1] Without loss of generality, assume the adversary controls the first t parties with private inputs $\{x_i\}_{i=1}^{t}$, and that the public statements are (g, y) with challenge d honestly sampled.

- **Case 1: Hash-based commitments.** Suppose the commitment scheme is defined as $\mathsf{C}_i := \mathsf{H}(f(i), r(i))$. The simulator proceeds as follows:
  1. Sample a random degree-t polynomial $z'(X)$ and define $\mathsf{C}'_0 := g^{z'(0)}y^d$.
  2. Sample another random degree-t polynomial $f'(X)$ such that $f'(i) = x_i$ for all $i = 1, \ldots, t$. Then, define $r'(X) := z'(X) + df'(X)$.
  3. For all $i = 1, \ldots, n$, set $\mathsf{C}'_i := \mathsf{H}(f'(i), r'(i))$.

  Finally, $\mathcal{S}$ outputs the transcript $(\mathsf{C}'_0, \mathsf{C}'_1, \ldots, \mathsf{C}'_n, d, z'(X))$, which is computationally indistinguishable from a real execution.

- **Case 2: Pedersen commitments.** Suppose $\mathcal{C}$ is the Pedersen commitment with two random generators (g, h), i.e., $\mathsf{C}_i := g^{f(i)}h^{r(i)}$. The simulator proceeds as follows:
  1. W.l.o.g., for $i = 1, \ldots, t$, compute $e_i := g^{x_i} = g^{f(i)}$.
  2. Using $y = g^{f(0)}$ together with $e_1, \ldots, e_t$, using the Lagrange interpolation in the exponent, interpolate to recover $e_i = g^{f(i)}$ for all $i = t+1, \ldots, n$.
  3. Assume the simulator knows the trapdoor $\alpha = \log_g h$.
  4. Sample a random degree-t polynomial $z'(X)$ and define $\mathsf{C}'_0 := g^{z'(0)}y^d$.
  5. Then. for each $i = 1, \ldots, n$, compute $\mathsf{C}'_i := e_i \cdot (g^{z'(i)}e_i^d)^{\alpha} = g^{f(i)}h^{z'(i)+df(i)}$.

  Finally, $\mathcal{S}$ outputs the transcript $(\mathsf{C}'_0, \mathsf{C}'_1, \ldots, \mathsf{C}'_n, d, z'(X))$, which is computationally indistinguishable from the real one.

In the non-interactive setting, the simulator additionally needs to program the random oracle to return the target challenge value. Since the protocol is public-coin, applying the Fiat–Shamir transform yields a non-interactive threshold ZK proof scheme in the random oracle model [6]. □

[1] One may generalize the theorem using equivocal commitments, thereby extending it to a broader class of commitment schemes; however, in this work we restrict our attention to the two cases mentioned above.

3.3 Construction of PCΠ Protocol

In this section, we present the construction of PCΠ, a DL-oriented and pre-constructed variant of the general Π framework [3]. The protocol PCΠ employs the DL-oriented NI-TZK proof system from Fig. 3 as a subroutine. This proof enables the dealer to convince the shareholders that the opened DL instance $y = g^{f(0)}$ is correctly derived from the secret value $f(0)$ and that each party has privately received a valid Shamir share of it.

The construction of PCΠ is shown in Fig. 4 and the description of the original Π can be obtained by omitting the highlighted terms. Similar to the original Π protocol, it relies on a (not necessarily homomorphic) commitment scheme $\mathcal{C}$ and a random oracle $\mathcal{H}$. We prove the security of PCΠ in the following theorem, and in the remainder of the paper we employ two instantiations of the protocol: one with a DL-based commitment scheme and one with a hash-based commitment.

Theorem 2 (A Framework for DL-Oriented PC-VSS Schemes). *If the commitment scheme $\mathcal{C}$ is both hiding and binding, and $\mathcal{H}$ is modeled as a random oracle, then the generic construction in Fig. 4 yields a secure DL-oriented PC-VSS scheme in the honest-majority setting. In particular, it satisfies the properties of correctness, verifiability, and simulatability (as defined in Section 3.1) against any computationally bounded adversary.*

Proof. The proof of this theorem is analogous to that of Theorem 1 and can be viewed as an extension of the argument in [3, Theorem 3.1] to incorporate pre-constructability. The security of the new PCΠ protocol follows directly from the properties of the underlying DL-oriented NI-TZK proof scheme, given in Figure 3 and established in Theorem 1. In particular:

- *Correctness* of PCΠ is inherited from the correctness of the DL-oriented NI-TZK scheme.
- *Simulatability* follows from the threshold ZK property. Recall that in Theorem 1 we explicitly constructed the simulator for two concrete commitment schemes. These are precisely the schemes that later we employ in the instantiations of the proposed PC-VSS construction PCΠ.
- *Verifiability* is implied by the special soundness of the DL-oriented NI-TZK proof, which reduces to the hardness of the discrete logarithm problem. We clarify that, in this case, the needed $t + 1$ valid shares for the extraction of $f(X)$, can be provided by any set of (honest and) qualified parties. □

3.4 Instantiations of PCΠ

In our proposed DKG protocols, we employ two concrete instantiations of the new PCΠ protocol (given in Fig. 4). In the first instantiation, the commitment scheme $\mathcal{C}$ is set to the Pedersen commitment, i.e., $c_i = g_1^{f(i)} g_2^{r(i)}$ for random generators g_1, g_2. In the second instantiation, $\mathcal{C}$ is defined using a hash-based commitment, i.e., $c_i = \mathsf{H}(f(i), r(i))$ where H is a secure hash function. These instantiations can be seen as the DL-oriented and pre-constructed variants of the standard VSS schemes Π_{F} and Π_{LA}, originally built from the Π protocol and proposed in [3]. We denote the new variants by $\mathrm{PC}\Pi_{\mathrm{F}}$ and $\mathrm{PC}\Pi_{\mathrm{LA}}$.

Initial: Parties agree on public parameters pp, which includes the group generator g, the random oracle $\mathcal{H}$, and the parameters for commitment scheme $\mathcal{C}$.

Share: Given (n, t, pp), to share a random secret f_0, the dealer D proceeds as follows:

1. Sample a uniformly random polynomial $f(X)$ and $r(X)$ of degree t with coefficients in a ring $\mathbb{Z}_N$ (or a field $\mathbb{Z}_q$), subject to $f(0) = f_0$.
2. For $i = 0, 1, \cdots, n$: set $f_i := f(i)$, and $r_i := r(i)$.
3. Set $y = g^{f_0}$, $c_0 = g^{r_0}$, and $c_i = \mathcal{C}(f_i, r_i)$, for $i = 1, 2, \cdots, n$.
4. Set $z(X) = r(X) + d \cdot f(X)$ and $\pi_{share} := (y, c_0, \ldots, c_n, z(X))$, where d is the challenge obtained from the random oracle, i.e., $d := \mathcal{H}(y, c_0, \ldots, c_n)$;
5. Send share f_i securely to P_i and broadcast π_{share}.

Verify: Given $\pi_{share} := (y, c_0, c_1, \ldots, c_n, z(X))$, and the individual shares:

1. Each party P_i first checks if $z(X)$ is a degree t polynomial, and then computes $d := \mathcal{H}(y, c_0, c_1, \ldots, c_n)$ and uses his/her share f_i and checks if $g^{z(0)} = c_0 y^d$ and $c_i = \mathcal{C}(f_i, z(i) - d \cdot f_i)$. If the verification of P_i fails, then P_i broadcasts a complaint against the dealer D.
2. If the number of complaints against D exceeds the value t, the dealer will be disqualified, and the verification will result in a `false` outcome.
3. In case a shareholder P_i raises a complaint about the verification of their part, the dealer will broadcast $f_i = f(i)$ to enable everyone to verify it using the verification equation. If the verification passes, the protocol continues as usual. However, if it fails, the dealer will be disqualified, leading to a `false` verification outcome. Since the disqualification decision is solely based on the information broadcasted, all honest shareholders will ultimately reach a consensus either on a set of qualified parties $Q \subseteq \{P_1, P_2, \cdots, P_n\}$ or on rejecting the final verification.

Reconstruct: Reconstruction can be done by two approaches:

1. (Optimistic) $\mathsf{Reconstruct}^{opt}(g, y, f_0) \rightarrow \{f_0, \mathsf{false}\}$: Given g, y, and the opening f_0, it checks if $y = g^{f_0}$. If so, it returns f_0; otherwise it returns `false`.
2. (Pessimistic) $\mathsf{Reconstruct}^{pes}(\{f_i\}_{i \in Q, |Q| \geq t+1}, \pi_{share}) \rightarrow \{f_0, \mathsf{false}\}$: Given $\pi_{share} := (y, c_0, c_1, \ldots, c_n, z(X))$, each party P_i broadcasts the secret value f_i. A party P_i is said to be confirmed if $c_i = \mathcal{C}(f_i, z(i) - d \cdot f_i)$, where $d := \mathcal{H}(y, c_0, c_1, \ldots, c_n)$. Consider f_i values of any $t + 1$ confirmed parties and interpolate $f(X)$ of degree t that passes through those points. Finally, the output is $f_0 = f(0)$ or `false` (if $t + 1$ valid shares were not obtained).

Fig. 4. PCΠ: a DL oriented variant of Π framework for building pre-constructed VSS schemes. In the general construction, $\mathcal{H}$ denotes an instantiation for the random oracle, and $\mathcal{C}$ is a hiding and binding commitment scheme. The original Π protocol [3] can be obtained omitting the highlighted terms.

4 The GJKR Fully-Secure DKG Protocol Revisited

In this section, we introduce our first Distributed Key Generation (DKG) protocol, denoted by DKG_1, which guarantees that the resulting public key is perfectly uniform even in the presence of up to t malicious parties. The design of DKG_1 is inspired by the well-known GJKR DKG protocol [12,13], but achieves asymptotically better efficiency. In the GJKR protocol, the first phase relies on Pedersen's perfectly simulatable VSS scheme [17] to distribute a secret, while in the second

phase the qualified parties employ Feldman VSS [11] together with their initial shares to derive the final public key. Following this blueprint, in the initial phase DKG_1, we employ the perfectly simulatable Π_{P} VSS scheme [3] (reviewed in Section 2.2). This prevents rushing adversaries from "meaningfully" modifying the set of qualified parties in the next step—and consequently biasing the final public key—by ensuring they learn no information about others' shares. In the second phase, the qualified parties use our proposed computationally secure PC-VSS scheme, $\mathrm{PC}\Pi_{\mathrm{LA}}$ (from Section 3.4), along with their initial shares, to efficiently extract the final public key. As in several well-established DKG protocols [13,15], DKG_1 ensures that at the end of the protocol, the parties obtain public DL-based commitments to the shares of other parties. Achieving this property solely with hash-based VSS schemes—such as the original Π_{LA} [3] or the ABCP VSS scheme [1]—is non-trivial, since commitments made via hash functions cannot be aggregated.

Technical overview. In the first phase of DKG_1, each party P_i secret-shares a value $x_i = f^{(i)}(0)$ using the Π_{P} VSS scheme. Thanks to the perfect simulatability of Π_{P}, an adversary cannot make the secret-shared value $x_j = f^{(j)}(0)$ of a corrupted party P_j depend on the values shared by any honest parties. Moreover, for every non-disqualified party P_i, there exists a unique polynomial $f^{(i)}(X)$ that is recoverable by the honest parties, even if P_i later misbehaves. At the end of this IT-secure VSS stage, each honest party P_i holds the share $f^{(j)}(i)$ of every non-disqualified party P_j. Using an IT-secure VSS guarantees that the global secret x is fixed at the end of the first phase, and no subsequent misbehavior can alter its value. If a non-disqualified party misbehaves in later stages, its contribution x_j can be publicly reconstructed by the honest parties and injected into the public key computation. This property is crucial: it ensures that the final output x (and the corresponding public key $y = g_1^x$) remains unbiased, thereby enabling the construction of a fully secure and more efficient DKG protocol in the DL setting.

Once x is fixed, the qualified parties proceed to compute g_1^x efficiently and securely. To this end, each party P_i publishes $y_i = g_1^{f^{(i)}(0)}$ and re-shares $f^{(i)}(0)$ using the new PC-VSS scheme $\mathrm{PC}\Pi_{\mathrm{LA}}$. Importantly, the same polynomial $f^{(i)}(X)$ used in the IT-secure VSS stage is employed in this PC-VSS stage, ensuring consistency. This enables P_i to prove that its public value $y_i = g^{f^{(i)}(0)}$ corresponds to the same secret that was shared earlier. Consequently, the new proof π_{share} published in second phase is verified using the shares $\{x_{ij}\}_{j=1}^n$ that were already distributed and validated during the initial IT-secure VSS stage.

By requiring each party to open its public key contribution $y_i = g_1^{f^{(i)}(0)}$ (for $i \in Q$) and verify it through the $\mathrm{PC}\Pi_{\mathrm{LA}}$ proof using the same IT-secure shares, we guarantee consistency across both phases. This prevents adversaries from altering their contributions during the public key computation. If a party P_i acts maliciously –for instance, by refusing to publish π_{share} or by executing it incorrectly– the honest parties can fall back on the reconstruction procedure of Π_{P} to recover $x_i = f^{(i)}(0)$. This ensures that P_i's contribution is still in-

DKG_1

VSS Stage:

1. Given random group generators (g_1, g_2, g_3), the parameters (n, t), and random oracle $\mathcal{H}$, each party P_i performs a Π_{P} VSS of a random value $f_0^{(i)}$ as a dealer:
   (a) P_i sample two uniformly random degree-t polynomials $f^{(i)}(X)$ and $r^{(i)}(X)$ with coefficients in $\mathbb{Z}_q$, subject to $f^{(i)}(0) = f_0^{(i)}$. For $j \in [n]$: set $f_{ij} := f^{(i)}(j)$, and $r_{ij} := r^{(i)}(j)$. For $j \in [n]$: set $c_j^{(i)} = g_1^{f_{ij}} g_2^{r_{ij}} g_3^{\gamma_{ij}}$, where γ_{ij} are random elements from $\mathbb{Z}_q$. Compute the challenge value $d_i := \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$; Check if $g_1 \neq g_2^{d_i}$ and if the check passed, set $z_i(X) = r^{(i)}(X) + d_i \cdot f^{(i)}(X)$ and $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z_i(X))$. Otherwise, sample a new values for γ_{ij} and recomputes the commitments; Finally, send the shares (f_{ij}, γ_{ij}) securely to party P_j and broadcast $\pi_{share}^{(i)}$.
   (b) Given, the public parameters and proofs $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z_i(X))$ for $i \in [n]$, and the shares $\{f_{ij}, \gamma_{ij}\}_{i=1}^n$, party P_j verifies the shares he received from other parties. To this end, for $i \in [n]$, P_j first checks if $z_i(X)$ is a degree-t polynomial, computes challenge $d_i = \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$ and then uses his/her shares (f_{ij}, γ_{ij}) and checks if
   $$c_j^{(i)} == g_1^{f_{ij}} g_2^{z_i(j) - d_i \cdot f_{ij}} g_3^{\gamma_{ij}}. \tag{3}$$
   If the check fails for an index i, then P_j broadcasts a *complaint* against P_i.
   (c) Each party P_i who, as a dealer, received a complaint from party P_j broadcasts the values (f_{ij}, γ_{ij}) that satisfy the above verification equation.
   (d) Each party marks as *disqualified* any party that either
   - received more than t complaints in Step 1.(b), or
   - answered a complaint in Step 1.(c) with values that falsify the check.
2. Each party then builds the set of non-disqualified parties QUAL.
3. The distributed secret value x is not explicitly computed by any party, but it equals $x = \sum_{i \in \mathsf{QUAL}} f_0^{(i)}$. Each party P_i sets his share of the secret as $x_i = \sum_{j \in \mathsf{QUAL}} f_{ji}$ and the public commitments (to others' share) as $c'_j = \prod_{i \in \mathsf{QUAL}} c_j^{(i)}$ for $j = 1, \ldots, n$.

Public Key Computation (Extracting $y = g_1^x$):

4. Each party $i \in \mathsf{QUAL}$ exposes $y_i = g_1^{f_0^{(i)}}$ and runs the PC-VSS scheme $\mathrm{PC}\Pi_{\mathrm{LA}}$, given in Fig. 4, using the same secret polynomial $f^{(i)}(X)$ from the VSS stage, and obtains a π_i.
   (a) Each P_i, $i \in \mathsf{QUAL}$ publish $\pi_i := (g_1, y_i = g_1^{f_0^{(i)}}, \mathsf{C}_0^{(i)}, \mathsf{C}_1^{(i)}, \ldots, \mathsf{C}_n^{(i)}, z'_i(X))$.
   (b) Each P_j verifies the values broadcast by the other parties in QUAL, namely, for each $i \in \mathsf{QUAL}$, P_j sets $d'_i \leftarrow \mathcal{H}(g_1, y_i, \mathsf{C}_0^{(i)}, \mathsf{C}_1^{(i)}, \ldots, \mathsf{C}_n^{(i)})$ and checks if
   $$\mathsf{C}_j^{(i)} == \mathsf{H}(f_{ij}, z'_i(j) + d'_i \cdot f_{ij}) \ \wedge \ \mathsf{C}_0 == g^{z'_i(0)} y_i^{d'_i}. \tag{4}$$
   If the check fails for an index i, P_j complains against P_i by broadcasting the values (f_{ij}, γ_{ij}) that satisfy (3) but do not satisfy (4).
   (c) For parties P_i who receive at least one valid complaint, i.e., values which satisfy (3) and not (4), the other parties run the reconstruction phase of Π_{P} VSS scheme to compute $f_0^{(i)}$ and $f^{(i)}(X)$ in the clear. For all parties in QUAL, set $y_i = g_1^{f_0^{(i)}}$. Compute $y = g_1^x = \prod_{i \in \mathsf{QUAL}} y_i$ and finally return y and c'_j for $j \in \mathsf{QUAL}$.

Fig. 5. DKG_1, a fully-secure distributed key generation in the honest majority setting.

cluded in the final key x, since excluding it could allow the adversary to bias its distribution.

We present our first DKG protocol for the DL setting, DKG_1, shown in Figure 5. Its security guarantees are formally proven in the theorem that follows.

Theorem 3. *Under the DL assumption, the protocol in Fig. 5 is a secure DKG protocol in the random oracle model, namely it satisfies the correctness and perfect secrecy properties against a malicious adversary corrupting up to t parties, with $t < n/2$.*

Simulator $\mathcal{S}_1$

Let $\mathcal{T}$ be the set of parties controlled by the adversary and $\mathcal{U}$ be the set of honest parties (run by the simulator). Without loss of generality, $\mathcal{T} = \{1, \ldots, t'\}$ and $\mathcal{U} = \{t'+1, \ldots, n\}$, with $t' \leq t$.

Input: public key y

1. Run Steps 1–3 on behalf of the non-corrupted parties $P_{t'+1}, \ldots, P_n$ exactly as in protocol DKG_1. This includes receiving and processing the information sent privately and publicly from corrupted parties to honest ones. At the end of Step 2 the following hold:
   - The set QUAL is well-defined. Note that $\mathcal{U} \subseteq \mathsf{QUAL}$ and that polynomials $f_i(X), r_i(X)$ for $i \in \mathcal{U}$ are chosen at random.
   - The adversary's view consists of polynomials $f^{(i)}(X), r^{(i)}(X)$ for $i \in \mathcal{T}$, γ_{ij} for $i \in \mathcal{T}, j \in [n]$, the shares $\left(f^{(i)}(j), \gamma_{ij}\right)$ for $i \in \mathsf{QUAL}, j \in \mathcal{T}$, and all the public values $\left(c_1^{(i)}, \ldots, c_n^{(i)}\right)$ and public polynomial z_i for $i \in \mathsf{QUAL}$.
   - $\mathcal{S}_1$ knows all polynomials $f^{(i)}(X), r^{(i)}(X)$ for $i \in \mathsf{QUAL}$. Note that for $i \in \mathsf{QUAL} \cap \mathcal{T}$, the honest parties, and hence $\mathcal{S}_1$, receive enough consistent shares from the adversary that allow $\mathcal{S}_1$ to compute all these parties' polynomials.
2. Perform the following computations:
   (a) Compute $y_i = g^{f^{(i)}(0)}$ for $i \in \mathsf{QUAL} \setminus \{n\}$.
   (b) Set $y_n^* = y \cdot \prod_{i \in (\mathsf{QUAL} \setminus \{n\})} (y_i)^{-1} \bmod q$.
   (c) Sample a random d'_n and a random degree-t polynomial $z'_n(X)$, and set $\mathsf{C}_0^{(n)} = g^{z'_n(0)} (y_n^*)^{d'_n}$. Sample another random degree-t polynomial $f^{*(n)}(X)$ such that $f^{*(n)}(j) = f^{(n)}(j)$ for $j \in \mathcal{T}$.
   (d) For $j \in [n]$, set $\mathsf{C}_j^{(n)} = \mathsf{H}(f^{*(n)}(j), z'_n(j) + d'_n \cdot f^{*(n)}(j))$.
   (e) Program random oracle $\mathcal{H}$ such that $\mathcal{H}(g, y_n^*, \mathsf{C}_0^{(n)}, \ldots, \mathsf{C}_n^{(n)}) = d'_n$ and set $\pi_n^* = (g, y_n^*, \mathsf{C}_0^{(n)}, \ldots, \mathsf{C}_n^{(n)}, z'_n(X))$.
3. Perform Step 4 of the protocol for party P_j, $j \in \mathcal{U}$.
   (a) Broadcast π_i for $i \in \mathcal{U} \setminus \{n\}$, and π_n^*.
   (b) Perform for each uncorrupted party the verifications of Eq. (4) on the values $\pi_i, i \in \mathcal{T}$, broadcast by the parties controlled by the adversary. If the verification fails for some $i \in \mathcal{T}, j \in \mathcal{U}$, broadcast a complaint (f_{ij}, γ_{ij}). (Notice that the corrupted parties can publish a valid complaint only against one another.)
   (c) Perform Step 4(c) of the protocol on behalf of the uncorrupted parties, i.e., perform reconstruction phase of Π_{P} VSS to compute $f_0^{(i)}$ and $f^{(i)}(X)$ in the clear, for every P_i against whom a valid accusation was broadcast in the previous step.

Fig. 6. Simulator $\mathcal{S}_1$ for the DKG protocol in Fig. 5.

Proof. **Correctness:** (C1) follows from the Verifiability and Reconstructibility properties of Π_{P} VSS scheme used in the VSS stage. Each party can tell apart the correct shares from the incorrect ones. Also, if an honest party P_i is not disqualified, then it holds that $x_i = \sum_{j \in \mathsf{QUAL}} f_{ji} = \sum_{j \in \mathsf{QUAL}} f^{(j)}(i)$, where $f^{(j)}(X)$ is the unique polynomial that is recoverable by the honest parties at the end of the VSS stage. Moreover, since $f^{(j)}(X)$ is a degree-t polynomial with $f^{(j)}(0) = f_0^{(j)}$, it holds that $x = \sum_{i \in \mathsf{QUAL}} f_0^{(i)}$. (C2) follows from the correctness of $\mathrm{PC}\Pi_{\mathrm{LA}}$scheme used in the public key computation stage. (C3) follows from the fact the honest parties pick random $f_0^{(i)}$ and the VSS is perfectly simulatable. Therefore, an adversary cannot learn any information about the shares of honest parties about the final key x. □

Perfect secrecy: Without loss of generality, we assume that the adversary corrupts the parties $P_1, \ldots, P_{t'}$, where $t' \leq t$. We denote the set of indices of corrupted parties by $\mathcal{T} := \{1, \ldots, t'\}$ and the set of indices of uncorrupted parties by $\mathcal{U} := \{t'+1, \ldots, n\}$. We provide a simulator $\mathcal{S}_1$ in Fig. 6, and we show that

the view of the adversary $\mathcal{A}$ that interacts with $\mathcal{S}_1$ on input y is identical to the view of $\mathcal{A}$ that interacts with the honest parties in a regular run of the protocol that outputs y as the public key. In a regular run of the protocol, $\mathcal{A}$ observes the following elements:

$$\left\{\left\{f^{(i)}(j),\gamma_{ij}\right\}_{j\in\mathcal{T}},\left\{c_k^{(i)},\mathsf{C}_k^{(i)}\right\}_{k\in[n]},z_i,z_i',y_i\right\}_{i\in\mathcal{U}}$$

in addition to elements that $\mathcal{A}$ itself outputs via the corrupted parties in $\mathcal{T}$. In an honest execution of the protocol, the view is random subject to the following conditions:

$$c_j^{(i)}=g_1^{f_{ij}}g_2^{z_i(j)-d_i\cdot f_{ij}}g_3^{\gamma_{ij}},\quad \forall i\in\mathsf{QUAL},j\in[n]$$
$$\mathsf{C}_j^{(i)}=\mathsf{H}(f_{ij},z_i'(j)+d_i'\cdot f_{ij}),\quad \forall i\in\mathsf{QUAL},j\in[n]$$
$$\mathsf{C}_0^{(i)}=g^{z_i'(0)}y_i^{d_i'},\quad \forall i\in\mathsf{QUAL}$$
$$\textstyle\prod_{i\in\mathsf{QUAL}}y_i=y$$

Figure 6 shows that $\mathcal{S}_1$ perfectly enforces these conditions. $\mathcal{S}_1$ follows the honest execution of honest parties in the VSS stage, so the first condition is satisfied in the view of $\mathcal{A}$. Note that the honest execution ensures that $z_i(X)$, $f^{(i)}(X)$, γ_{ij}, and $c_j^{(i)}$, for $i\in\mathcal{U},j\in[n]$ are uniformly random. In the public key computation stage, for $i\in\mathcal{U}\backslash\{n\}$, $\mathcal{S}_1$ computes z_i' honestly which gives a uniform distribution. For $i=n$, as shown in Step 2 of Fig. 6, $\mathcal{S}_1$ picks a random degree-t polynomial $f^{*(n)}$ such that it is equal to $f^{(n)}$ on the indices in $\mathcal{T}$, so that it is consistent with the shares that $\mathcal{A}$ has seen in the VSS stage. Also, $\mathcal{S}_1$ picks y_n^* such that the product of all y_i is equal to y. Then, $\mathcal{S}_1$ picks a random degree-t polynomial z_n' and computes $\mathsf{C}_j^{(n)}$ for $j\in[n]$ and $\mathsf{C}_0^{(n)}$ as in the protocol. This ensures that the second and third conditions are satisfied. Finally, $\mathcal{S}_1$ programs the random oracle $\mathcal{H}$ to ensure that d_n' is consistent with the values it has chosen. Notice that in the public key computation stage, there is no homomorphic property that the adversary can exploit to understand that y_n^* and $f^{*(n)}(X)$ are not consistent. This concludes the proof of perfect secrecy. □

Efficiency of DKG_1. Comparing DKG_1 protocol to the fully-secure variant of Pedersen DKG presented in [12, 13], we observe asymptotic improvements in computational cost of parties. In DKG_1, each party is required to perform approximately $8n$ exponentiations within the group[2], conduct $5n$ evaluations of degree-t polynomials in the field, and execute $3n$ hash operations for commitment purposes. While, in the GJKR DKG each party needs to compute $(2tn+n)$ exponentiations in the group, $2n$ degree-t polynomial evaluations in the field, and a single hash operation. In terms of communications, our DKG protocol entails

[2] We note that by using $\Pi_{\mathrm{P}}+$ [2] instead of Π_{P} [3], one could construct a variant of our protocol that requires $6n$ exponentiations (instead of $8n$); however, in this case the commitments to the parties' shares would no longer be additively homomorphic, which might introduce challenges in the final applications.

each party privately sending $2n$ field elements to other participants, along with broadcasting approximately n images generated through hash functions and n group elements (i.e., the commitments) and $2t$ field elements (i.e., coefficients of polynomials). Conversely, in the GJKR DKG protocol, each party privately sends $2n$ field elements to the other participants and then broadcast $2t$ group elements. We refer to Table 1 for a summarized comparison of the asymptotic costs in both fully-secure DKG protocols.

5 Fully-Secure Computationally Uniform DKG

In this section, we introduce another fully secure and robust DKG protocol for sampling a DL instance, $y = g_1^x$, which we call DKG_2. Compared to DKG_1 (from Section 4), the protocol DKG_2 can be concretely more efficient in practice, though at the cost of guaranteeing that the distribution of the public key is only *computationally indistinguishable* from a random one (rather than perfectly indistinguishable) in the presence of up to t malicious parties. The design of DKG_2 follows the blueprint of DKG_1, but with two main modifications:

1. In the first phase, we replace the perfectly simulatable VSS scheme Π_{P} with the hash-based VSS scheme Π_{LA} [3] (reviewed in Sec. 2.2), which offers computational simulatability.
2. In the second phase, we replace the PC-VSS scheme $\mathrm{PC}\Pi_{\mathrm{LA}}$ with our other PC-VSS scheme $\mathrm{PC}\Pi_{\mathrm{F}}$ from Section 3.4.

Similar to $\mathrm{PC}\Pi_{\mathrm{LA}}$, the scheme $\mathrm{PC}\Pi_{\mathrm{F}}$ enables a dealer to prove that she knows a secret $f(0)$ such that $y = g^{f(0)}$, while simultaneously ensuring that every shareholder has received a valid Shamir share of $f(0)$. We note that in practice one could also use $\mathrm{PC}\Pi_{\mathrm{LA}}$ in the second phase of DKG_2, yielding an even more efficient DKG protocol in concrete terms. However, this comes with a trade-off: the resulting DKG would realize a slightly weaker ideal functionality, as it would provide DL-based commitments only to the *public key contributions* of the parties, rather than to their individual *shares*. In this work, we focus on the construction of fully secure DKG protocols which, similarly to Katz's protocols [15], additionally output DL-based commitments to the individual shares of the parties at the end of the protocol.

By relying on Π_{LA} in the first phase, which is fully hash-based and non-malleable, DKG_2 prevents computationally bounded (including quantum) adversaries from manipulating the set of qualified parties based on the values $f^{(i)}(0)$. In other words, during the VSS stage, a corrupted party P_j cannot force its secret value $x_j = f^{(j)}(0)$ to depend on the values of honest parties. As a result, DKG_2 guarantees that the distribution of the public key is *computationally uniform*, even in the presence of up to t quantum adversaries.

As in DKG_1, at the end of the VSS stage, every non-disqualified party P_i is associated with a unique polynomial $f^{(i)}(X)$, which can later be reconstructed by honest parties if needed (e.g., if P_i misbehaves). Thus, once the VSS stage is completed, the value of the secret x is fully determined and cannot be altered by

subsequent malicious behavior. Moreover, each honest party P_i holds the share $f^{(j)}(i)$ for every non-disqualified party P_j.

After fixing both the secret x and the set of qualified parties QUAL, the parties jointly compute the final public key $y = g_1^x$ in a way similar to DKG_1 (see Fig. 5), but this time using $\mathrm{PCII}_{\mathrm{F}}$. Each party $P_i \in \mathsf{QUAL}$ publishes its public contribution $y_i = g_1^{f^{(i)}(0)}$ and re-shares it with $\mathrm{PCII}_{\mathrm{F}}$ using the same secret polynomial from the VSS stage. Other parties then verify these contributions through the $\mathrm{PCII}_{\mathrm{F}}$ proof, re-using the shares from the VSS stage. This enforces consistency across both the VSS and public key computation phases, preventing adversaries from altering their contributions midway. Finally, if a party P_i misbehaves during this stage—for instance, by refusing to publish π_{share} or publishing an invalid one—the honest parties can invoke the reconstruction procedure of Π_{LA} to recover $x_i = f^{(i)}(0)$. This ensures that P_i's contribution is still included in the final key x, since excluding it could allow an adversary to bias the resulting distribution.

The full description of our second fully secure hash-based DKG protocol, DKG_2, is given in Fig. 7. Its security guarantees are formally established in the theorem that follows.

Theorem 4. *Under the DL assumption, the protocol in Fig. 7 is a secure DKG protocol in the random oracle model, namely it satisfies the correctness and computational secrecy properties against a malicious adversary corrupting up to t parties, with $t < n/2$.*

Proof. Similar to the proof of Theorem 3, the proof of coreectness and computational secrecy follows from the properties of underlying Π_{LA} VSS and $\mathrm{PCII}_{\mathrm{F}}$ PC-VSS schemes. □

Efficiency of DKG_2. As in DKG_1, comparing DKG_2 to the fully-secure GJKR protocol [12,13], we observe asymptotic improvements in computational cost of parties. In DKG_2, each party is required to perform approximately $4n$ exponentiations within the group, conduct $5n$ evaluations of degree-t polynomials in the field, and execute $5n$ hash operations for commitment and random oracle purposes. In terms of communications, our second DKG protocol entails each party privately sending n field elements to other participants, along with broadcasting approximately n images generated through hash functions and n group elements and $2t$ field elements (i.e., coefficients of polynomials $z(x)$ and $z'(x)$).

Our DKG_2 as a Fully Secure Alternative to the ABCP DKG. In [1], Atapoor, Baghery, Cozzo, and Pedersen introduced the first practical hash-based VSS protocol in the synchronous setting, which they then used to build an efficient DKG protocol for the DL setting with $O(n)$ computational complexity. Our fully secure hash-based DKG protocol, DKG_2, can be viewed as an alternative to their construction. However, in what follows, we explain why the ABCP DKG protocol does not achieve full security and show how an adversary can bias the distribution of the final public key.

VSS Stage:

1. Given the parameters (n, t), hash-based commitment scheme $\mathcal{C}$, and random oracle $\mathcal{H}$, each party P_i performs a Π_{LA} VSS of a *random* value $f_0^{(i)}$ as a dealer:
   (a) P_i sample two uniformly random degree-t polynomials $f^{(i)}(X)$ and $r^{(i)}(X)$ with coefficients in $\mathbb{Z}_q$, subject to $f^{(i)}(0) = f_0^{(i)}$. For $j = 1, 2, \cdots, n$: set $f_{ij} := f^{(i)}(j)$, and $r_{ij} := r^{(i)}(j)$. For $j = 1, 2, \cdots, n$: set $c_j^{(i)} = \mathsf{H}(f_{ij}, r_{ij})$. Compute the challenge value $d^{(i)} := \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$; Set $z^{(i)}(X) = r^{(i)}(X) - d^{(i)} \cdot f^{(i)}(X)$ and $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z^{(i)}(X))$. At the end, send share f_{ij} securely to P_j and return $\pi_{share}^{(i)}$ as the proof for correctness of the commitment and sharing.
   (b) Given, the public parameters and proofs $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z^{(i)}(X))$ for $i = 1, \ldots, n$, and the shares $\{f_{ij}\}_{i=1}^n$, party P_j verifies the shares and commitments he received from other parties. To this end, for $i = 1, \ldots, n$, P_j first checks if $z^{(i)}(X)$ is a degree-t polynomial, computes challenge value $d^{(i)} = \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$ and then uses his/her share f_{ij} and checks if

$$c_j^{(i)} == \mathsf{H}(f_{ij}, z^{(i)}(j) + d^{(i)} \cdot f_{ij}) \tag{5}$$

   If the check fails for an index i, then P_j broadcasts a *complaint* against P_i.
   (c) Each party P_i who, as a dealer, received a complaint from party P_j broadcasts the value f_{ij} that satisfy verification (5).
   (d) Each party marks as *disqualified* any party that either
   - received more than t complaints in Step 1.(b), or
   - answered a complaint in Step 1.(c) with values that falsify the check.
2. Each party then builds the set of non-disqualified parties QUAL. (We later show that all honest parties build the same set QUAL.)
3. The distributed secret value x is not explicitly computed by any party, but it equals $x = \sum_{i \in \mathsf{QUAL}} f_0^{(i)}$. Each party P_i sets his share of the secret as $x_i = \sum_{j \in \mathsf{QUAL}} f_{ji}$.

Public Key Computation (Extracting $y = g_1^x$):

4. Each party $i \in \mathsf{QUAL}$ publish $y_i = g_1^{f_0^{(i)}}$ and runs the PC-VSS scheme $\text{PC}\Pi_{\text{F}}$ in Fig. 4 and Section 3.4, using the same secret polynomial $f^{(i)}(X)$ from the VSS stage, and obtains a π_i.
   (a) Each P_i, $i \in \mathsf{QUAL}$ publish $\pi_i := (g_1, y_i = g_1^{f_0^{(i)}}, \mathsf{C}_0^{(i)}, \mathsf{C}_1^{(i)}, \ldots, \mathsf{C}_n^{(i)}, z_i'(X))$.
   (b) Each P_j verifies the values broadcast by the other parties in QUAL, namely, for each $i \in \mathsf{QUAL}$, P_j sets $d_i' \leftarrow \mathcal{H}(g_1, y_i, \mathsf{C}_0^{(i)}, \mathsf{C}_1^{(i)}, \ldots, \mathsf{C}_n^{(i)})$ and checks if

$$\mathsf{C}_j^{(i)} == g_1^{f_{ij}} g_2^{z_i'(j) + d_i' \cdot f_{ij}} \ \wedge \ \mathsf{C}_0 == g_1^{z_i'(0)} y_i^{d_i'}. \tag{6}$$

   If the check fails for an index i, P_j complains against P_i by broadcasting the value f_{ij} that satisfy (5) but do not satisfy (6).
   (c) For parties P_i who receive at least one valid complaint, i.e., values which satisfy (5) and not (6), the other parties run the reconstruction phase of Π_{LA} VSS scheme to compute $f_0^{(i)}$ and $f^{(i)}(X)$ in the clear, and also the commitments to their shares. For all parties in QUAL, set $y_i = g_1^{f_0^{(i)}}$. Compute and return the final public key $y = g_1^x = \prod_{i \in \mathsf{QUAL}} y_i$ and the commitments $\mathsf{C}_j' = \prod_{i \in \mathsf{QUAL}} \mathsf{C}_j^{(i)}$ for $j \in [n]$.

Fig. 7. DKG_2: A hash-based fully-secure distributed key generation for DL.

The DKG protocol of [1], presented in their Fig. 4, proceeds in two phases: the *VSS and Committing* phase and the *Opening, Verification, and Public Key Computation* phase. In the first phase, each party samples a random secret, shares it using their VSS scheme, and commits to its partial public key. In the second phase, each party opens its commitment and privately sends the corresponding shares to the others. Upon receiving the openings and proofs,

parties verify the validity of the received shares. If a party refuses to open its commitment at the start of the second phase, it is disqualified from the protocol.

This design, however, creates an opening for a rushing adversary. By waiting to see the openings of honest parties before deciding whether to open its own commitment, the adversary can selectively abort and thereby bias the distribution of the final public key. A well-known countermeasure, which we also adopt in our proposed protocols, is to ensure that adversaries cannot make such abort decisions after learning any information about other parties' partial public keys. This requires that verification of the distributed shares must take place before exposing any information that could leak partial public keys to corrupted parties.

Table 1 summarizes the asymptotic costs of our fully secure hash-based protocol DKG_2 compared to the biased ABCP DKG protocol [1].

6 DKG_3: A Round-Optimized Variant of DKG_2

In this section, we present DKG_3 as a round optimized variant of our fully secure and robust DKG protocol DKG_2 from Section 5. Compared to DKG_2 that required 6 rounds in the general case (and 2 rounds in the happy path, where all parties follow the protocol), DKG_3 requires 3 rounds in general case, and slightly less communications, but at the cost of slightly increased computational costs. Similar to DKG_2, DKG_3 ensures that the distribution of the generated public key is computationally indistinguishable in the presence of up to t adversaries.

Technical Overview. To construct DKG_3, we utilize two key sub-protocols: the round-optimal variant of the Π_{F} VSS scheme from [4] (referred to as 2R-Π_{F} and given in Fig. 1), and an efficient NIZK argument of knowledge for the relation (1), reviewed in Section 2.4. The round-optimal scheme 2R-Π_{F} allows a party P_i to share a secret $f^{(i)}(0)$ in two rounds such that, at the end of the protocol, each party P_j receives the share $f^{(i)}(j)$ for $j \in [n]$. To achieve this, 2R-Π_{F} employs the following technique. In the first round, parties publish commitments to one-time secret keys and send the corresponding secret keys privately to the dealer. In the second round, the dealer uses these secret keys to broadcast one-time-pad encryptions of the shares. The commitments published by the parties and the ciphertexts broadcast by the dealer enable the parties to resolve potential conflicts in the rest of protocol without requiring additional interaction or rounds.

Once the VSS stage finished, the value of x will be fixed, the qualified set QUAL will be determined, and each party P_i will publish a set of commitments $c_j^{(i)} = g_1^{f_{ij}} g_2^{r_{ij}}$ for $j \in [n]$, as the part of his π_{share}. Then, the protocol proceeds to the single-round public key computation phase in which all parties in QUAL are required to invoke the NIZK argument of knowledge from Fig. 2. This argument enables a party to publicly and in a zero-knowledge manner convince the verifiers that the commitment

$$c_i' := \prod_{j \in \mathsf{QUAL}} c_j^{(i)} = g_1^{\sum_{j \in \mathsf{QUAL}} f_{ij}} g_2^{\sum_{j \in \mathsf{QUAL}} r_{ij}}$$

and the group element $y_i \in \mathbb{G}$ encode the same exponent $x_i := \sum_{j\in\mathsf{QUAL}} f_{ij}$; that is, $c'_i = g_1^{x_i} g_2^{x'_i}$ and $y_i = g_3^{x_i}$, where (g_1, g_2, g_3) are independently sampled group generators. This proof enables a party $P_i \in \mathsf{QUAL}$ to convince all other parties, in a *single round*, that its opened value $y_i = g_3^{x_i}$ is valid and has been computed

2-Round VSS Stage:

1. Given random group generators (g_1, g_2, g_3), the parameters (n, t), and random oracle $\mathcal{H}$, each party P_i performs a 2R-Π_{F} VSS of a *random* value $f_0^{(i)}$ as a dealer:
   (a) P_i sample two uniformly random degree-t polynomials $f^{(i)}(X)$ and $r^{(i)}(X)$ with coefficients in a field $\mathbb{Z}_q$, subject to $f^{(i)}(0) = f_0^{(i)}$. For $j = 1, 2, \cdots, n$: set $f_{ij} := f^{(i)}(j)$, and $r_{ij} := r^{(i)}(j)$. For $j \in [n]$: set $c_j^{(i)} = g_1^{f_{ij}} g_2^{r_{ij}}$. Compute the challenge value $d^{(i)} := \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$; Set $z^{(i)}(X) = r^{(i)}(X) - d^{(i)} \cdot f^{(i)}(X)$. For $j \in [n]$, $j \neq i$: picks random pairs $(s_{ij}, t_{ij}) \in \mathbb{Z}_q$, and sets $c_{s_{ij}} = \mathsf{H}(s_{ij}, t_{ij})$. Then, set $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z^{(i)}(X), c_{s_{i1}}, \ldots, c_{s_{in}})$. At the end, send secret values (s_{ij}, t_{ij}) and share f_{ij} securely to P_j and return $\pi_{share}^{(i)}$ as the proof for correctness of the commitment and sharing.
   (b) Given, the public parameters, each party P_i, checks if for $j \in [n], j \neq i$ the commitments $c_{s_{ji}}$ are consistent with the pairs (s_{ji}, t_{ji}), received from other parties P_j. Broadcast $\alpha_{ij} = f_{ij} + s_{ji}$ if the verification succeeds, and broadcast f_{ij} otherwise.
   (c) Given, the public parameters and proofs $\pi_{share}^{(i)} := (c_1^{(i)}, \ldots, c_n^{(i)}, z^{(i)}(X), c_{s_{i1}}, \ldots, c_{s_{in}})$ for $i = 1, \ldots, n$, and the shares $\{f_{ij}\}_{i=1}^{n}$, party P_j verifies the shares and commitments he received from other parties. To this end, for $i = 1, \ldots, n$, P_j first checks if $z^{(i)}(X)$ is a degree-t polynomial, computes challenge value $d^{(i)} = \mathcal{H}(c_1^{(i)}, \ldots, c_n^{(i)})$ and then uses his/her share f_{ij} and checks if

$$c_j^{(i)} == g_1^{f_{ij}} g_2^{z^{(i)}(j) + d^{(i)} \cdot f_{ij}} \tag{7}$$

   Broadcasts nothing if the verification succeeds, and broadcasts pairs (s_{ji}, t_{ji}) otherwise. Party P_j is considered happy in the former case, and unhappy in the later case.

2. **Local Computations:** Honest parties perform the local computations as in Fig. 1 and either abort the protocol or determine the set of non-disqualified parties QUAL. The distributed secret value x is not explicitly computed by any party, but it equals $x = \sum_{i\in\mathsf{QUAL}} f_0^{(i)}$. Each party P_i sets his share of the secret as $x_i = \sum_{j\in\mathsf{QUAL}} f_{ji}$ and also computes the commitment to the shares of all parties as $c'_i = \prod_{j\in\mathsf{QUAL}} c_j^{(i)}$ for $i \in \mathsf{QUAL}$.

Public Key Computation (Extracting $y = g_3^x$):

3. Each party $i \in \mathsf{QUAL}$ computes $y_i = g_3^{\sum_{j\in\mathsf{QUAL}} f_{ij}}$ and given $c'_i := \prod_{j\in\mathsf{QUAL}} c_j^{(i)} = g_1^{\sum_{j\in\mathsf{QUAL}} f_{ij}} g_2^{\sum_{j\in\mathsf{QUAL}} r_{ij}}$ and runs the NIZK proof scheme given in Fig. 2 with statement $(g_1, g_2, g_3, c'_i, y_i)$ and witness $(x_i := \sum_{j\in\mathsf{QUAL}} f_{ij}, x'_i := \sum_{j\in\mathsf{QUAL}} z^{(j)}(i) + d^{(j)} f_{ij}))$ and obtains a proof π_i^{PK} for the relation given in Eq. (1).
   (a) Each P_i, $i \in \mathsf{QUAL}$ publish the partial public key y_i and the proof $\pi_i^{PK} := (e_i, z_{1i}, z_{2i})$.
   (b) Each P_j verifies the values broadcast by the other parties in QUAL, namely, for each published party i, P_j checks if challenge value

$$e_i == \mathcal{H}(c'_i, y_i, \frac{g_1^{z_{1i}} g_2^{z_{2i}}}{(c'_i)^{e_i}}, \frac{g_3^{z_{1i}}}{(y_i)^{e_i}}). \tag{8}$$

   Let $\mathsf{QUAL}' \subseteq \mathsf{QUAL}$ denote the set of qualified parties that have published valid proofs. Finally, each party P_i, for $i \in \mathsf{QUAL}'$, locally computes the final public key

$$y = g_3^x = \prod_{i\in\mathsf{QUAL}'} y_i^{L_{0,i}},$$

   where $L_{0,i} := \prod_{j\in\mathsf{QUAL}'\setminus\{i\}} \frac{j}{j-i}$ are the Lagrange basis polynomials evaluated at 0. Each party also stores the set of $\{c'_i\}_{i\in\mathsf{QUAL}}$ as the commitments to the parties' shares.

Fig. 8. DKG_3: An efficient 3-round fully-secure DKG protocol for DL.

using sum of the same shares received during the VSS stage. Since the qualified set QUAL contains at least $t+1$ honest parties, the protocol guarantees that the honest parties obtain a sufficient number of valid proofs and group elements to reconstruct the final public key using the contributions of all parties in QUAL. Finally, the public key is computed locally with respect to the generator g_3 by performing Lagrange interpolation in the exponent over the valid group elements $\{y_i\}_{i\in\mathsf{QUAL}'}$ published by the parties. As in previous fully-secure DKG protocols, this ensures that all parties' contribution to the secret key x will still be included in the computation.

In terms of security, under the Diffie–Hellman assumption, computing $y_i = g_3^{F_i}$ from $y_i' = g_1^{F_i} g_2^{R_i}$ is computationally hard. Moreover, we show that when the shared secrets are chosen uniformly at random, the distribution of the final public key is uniform from the perspective of any computationally bounded adversary that corrupts up to t parties.

The description of new 3-round fully-secure DKG protocol DKG_3 is given in Fig. 8 and its security is established in the following theorem.

Theorem 5. *Under the DL assumption, the protocol in Fig. 8 is a secure DKG protocol in the random oracle model, namely it satisfies the correctness and computational secrecy properties against a malicious adversary corrupting up to t parties, with $t < n/2$.*

Proof. Similar to the proof of Theorem 3, correctness and computational secrecy follow from the security guarantees of the underlying 2R-Π_{F} VSS scheme together with the properties of the NIZK argument given in Fig. 2. We emphasize that, in this case, the general DKG simulator has access to the relations among the group generators (g_1, g_2, g_3) and uses this information as a trapdoor to simulate the protocol transcript. □

Efficiency of DKG_3. When compared to the fully secure GJKR protocol [13] and the recent DKG protocols by Katz [15], Cascudo and David [9], and Boneh et al. [7] which require totally 3 rounds, our new 3-round DKG protocol DKG_3 achieves asymptotically lower computational costs for all the parties.

A detailed summary of the asymptotic complexities of DKG_3 and other relevant DKG protocols is provided in Table 1.

7 Conclusion

We revisited the fully secure and robust variant of the Pedersen DKG protocol proposed by Gennaro, Jarecki, Krawczyk, and Rabin [13], and presented several new variants that achieve improved efficiency while offering different trade-offs in terms of communication rounds, public-key distribution, and computational and communication costs. As in the original GJKR protocol, our first fully secure and robust DKG construction guarantees that the final public key is distributed uniformly at random, even against a computationally unbounded adversary corrupting up to t parties. This strong distributional guarantee is a distinguishing feature that is not provided by many recent DKG protocols.

We further demonstrated that the recent practical hash-based robust DKG protocol of Atapoor, Baghery, Cozzo, and Pedersen [1] does not achieve full security, as an adversary can bias the distribution of the resulting public key.

Among our constructions, one DKG protocol requires only three online rounds and, in contrast to recent alternatives [7,9,14,15] that rely on one online round and two offline (or vice versa), achieves superior practical efficiency. In particular, the per-party computation is dominated by $O(n)$ exponentiations, rather than the $O(n^2)$ cost incurred by the most efficient existing schemes.

An interesting direction for future work is to further reduce the number of rounds in our three-round DKG protocol, or to shift some of its interaction into an offline or preprocessing phase, without sacrificing its asymptotic efficiency or strong security guarantees. Moreover, we believe that most of the techniques introduced in this work are sufficiently general to be adapted to the construction of post-quantum secure DKG protocols.

Acknowledgments

This work was supported by the Flemish Government through the Cybersecurity Research Program with grant number VOEWICS02.

References

1. Atapoor, S., Baghery, K., Cozzo, D., Pedersen, R.: VSS from distributed ZK proofs and applications. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023, Part I. Lecture Notes in Computer Science, vol. 14438, pp. 405–440. Springer, Singapore, Singapore, Guangzhou, China (Dec 4–8, 2023). https://doi.org/10.1007/978-981-99-8721-4_13
2. Atapoor, S., Baghery, K., Nicolas, G., Pedersen, R., Spiessens, J.: Batched and packed (publicly) verifiable secret sharing: A unified framework and applications. IACR Cryptol. ePrint Arch. p. 2018 (2025), https://eprint.iacr.org/2025/2018
3. Baghery, K.: Π: A unified framework for computational verifiable secret sharing. In: Jager, T., Pan, J. (eds.) PKC 2025: 28th International Conference on Theory and Practice of Public Key Cryptography, Part IV. Lecture Notes in Computer Science, vol. 15677, pp. 110–142. Springer, Cham, Switzerland, Røros, Norway (May 12–15, 2025). https://doi.org/10.1007/978-3-031-91829-2_4
4. Baghery, K., Bardeh, N.G., Khazaei, S., Rahimi, M.: On round-optimal computational VSS. IACR Communications in Cryptology **2**(2) (2025). https://doi.org/10.62056/a0zo-4tw9
5. Baghery, K., Knapen, N., Nicolas, G., Rahimi, M.: Pre-constructed publicly verifiable secret sharing and applications. In: Fischlin, M., Moonsamy, V. (eds.) ACNS 2025: 23rd International Conference on Applied Cryptography and Network Security, Part I. Lecture Notes in Computer Science, vol. 15825, pp. 89–119. Springer, Cham, Switzerland, Munich, Germany (Jun 23–26, 2025). https://doi.org/10.1007/978-3-031-95761-1_4
6. Boneh, D., Boyle, E., Corrigan-Gibbs, H., Gilboa, N., Ishai, Y.: Zero-knowledge proofs on secret-shared data via fully linear PCPs. In: Boldyreva, A., Micciancio, D.

(eds.) Advances in Cryptology – CRYPTO 2019, Part III. Lecture Notes in Computer Science, vol. 11694, pp. 67–97. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26954-8_3
7. Boneh, D., Haitner, I., Lindell, Y., Segev, G.: Exponent-vrfs and their applications. In: Fehr, S., Fouque, P. (eds.) Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15607, pp. 195–224. Springer (2025). https://doi.org/10.1007/978-3-031-91098-2_8, https://doi.org/10.1007/978-3-031-91098-2_8
8. Cascudo, I., Cozzo, D., Giunta, E.: Verifiable secret sharing from symmetric key cryptography with improved optimistic complexity. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 100–128. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). https://doi.org/10.1007/978-981-96-0941-3_4
9. Cascudo, I., David, B.: Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part V. Lecture Notes in Computer Science, vol. 14655, pp. 216–248. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58740-5_8
10. Chor, B., Goldwasser, S., Micali, S., Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults (extended abstract). In: 26th Annual Symposium on Foundations of Computer Science. pp. 383–395. IEEE Computer Society Press, Portland, Oregon (Oct 21–23, 1985). https://doi.org/10.1109/SFCS.1985.64
11. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th Annual Symposium on Foundations of Computer Science. pp. 427–437. IEEE Computer Society Press, Los Angeles, CA, USA (Oct 12–14, 1987). https://doi.org/10.1109/SFCS.1987.4
12. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: Stern, J. (ed.) Advances in Cryptology – EUROCRYPT'99. Lecture Notes in Computer Science, vol. 1592, pp. 295–310. Springer Berlin Heidelberg, Germany, Prague, Czech Republic (May 2–6, 1999). https://doi.org/10.1007/3-540-48910-X_21
13. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. Journal of Cryptology **20**(1), 51–83 (Jan 2007). https://doi.org/10.1007/s00145-006-0347-3
14. Kate, A., Mangipudi, E.V., Mukherjee, P., Saleem, H., Thyagarajan, S.A.K.: Non-interactive VSS using class groups and application to DKG. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024: 31st Conference on Computer and Communications Security. pp. 4286–4300. ACM Press, Salt Lake City, UT, USA (Oct 14–18, 2024). https://doi.org/10.1145/3658644.3670312
15. Katz, J.: Round-optimal, fully secure distributed key generation. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part VII. Lecture Notes in Computer Science, vol. 14926, pp. 285–316. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68394-7_10
16. Pedersen, T.P.: A threshold cryptosystem without a trusted party (extended abstract) (rump session). In: Davies, D.W. (ed.) Advances in Cryptology – EUROCRYPT'91. Lecture Notes in Computer Science, vol. 547, pp. 522–526. Springer

Berlin Heidelberg, Germany, Brighton, UK (Apr 8–11, 1991). https://doi.org/10.1007/3-540-46416-6_47
17. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) Advances in Cryptology – CRYPTO'91. Lecture Notes in Computer Science, vol. 576, pp. 129–140. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 11–15, 1992). https://doi.org/10.1007/3-540-46766-1_9
18. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) Advances in Cryptology – CRYPTO'89. Lecture Notes in Computer Science, vol. 435, pp. 239–252. Springer, New York, USA, Santa Barbara, CA, USA (Aug 20–24, 1990). https://doi.org/10.1007/0-387-34805-0_22
19. Shamir, A.: How to share a secret. Communications of the Association for Computing Machinery **22**(11), 612–613 (Nov 1979). https://doi.org/10.1145/359168.359176

Appendix:

A Feldman and Pedersen VSS Schemes

Feldman VSS Scheme. One of the primary computationally secure VSS schemes is Feldman's scheme, which is based on Shamir and was proposed in [11]. In Feldman's scheme, given (n, t, f_0) and group generator g_1, to share a *high-entropy* secret f_0, the Share and Verify algorithms are as follows:

- $\mathsf{Share}(n, t, f_0, g_1) \rightarrow (\{f_i\}_{i=1}^n, \pi_{share})$: Sample a uniformly random degree-t polynomial $f(X) := f_0 + a_1 X + \cdots + a_t X^t$ with coefficients in $\mathbb{Z}_q$, subject to $f(0) = f_0$. For $i = 1, 2, \cdots, n$: set $f_i := f(i)$. Compute $c_0 = g_1^{f_0}$ and $c_j = g_1^{a_j}$ for $j = 1, 2, \cdots, t$. Set $\pi_{share} := (c_0, c_1, \ldots, c_t)$; Sends share f_i securely to party P_i and return π_{share} as the NI-TZK proof.
- $\mathsf{Verify}(n, t, \{f_i\}_{i=1}^n, \pi_{share}) \rightarrow$ `true/false`: Given n, threshold value t, the shares $\{f_i\}_{i=1}^n$, and the threshold proof $\pi_{share} := (c_0, c_1, \ldots, c_t)$: each party P_i uses his/her share f_i and checks if

$$g_1^{f_i} = \prod_{j=0}^{t} c_j^{i^j}$$

  and outputs either `true` or `false`. For $n \geq 2t + 1$, if *all* n parties return `true`, then the final Verify will return `true`. Otherwise, any possible conflict between the dealer and the parties will be solved using a known conflict resolution approach used in our protocols as well.

Pedersen VSS Scheme. The Pedersen VSS scheme is a variation of Feldman's scheme [11] that uses a perfectly hiding commitment scheme. Pedersen VSS scheme ensures that fewer than t parties receive no information about the secret, thereby achieving Information-Theoretical (IT) simulatability. In Pedersen's scheme, given (n, t, f_0) and two random group generators $pp := (g_1, g_2)$, to share a secret f_0, the Share and Verify algorithms are as follows:

- $\mathsf{Share}(n, t, f_0, pp) \rightarrow (\{f_i\}_{i=1}^{n}, \pi_{share})$: Sample two random degree-t polynomials $f(X) := f_0 + a_1X + \cdots + a_tX^t$ and $r(X) := r_0 + b_1X + \cdots + b_tX^t$ with coefficients in $\mathbb{Z}_q$, subject to $f(0) = f_0$. For $i = 1, 2, \cdots, n$: set $f_i := f(i)$ and $r_i := r(i)$. Compute $c_0 = g_1^{f_0} g_2^{r_0}$ and $c_j = g_1^{a_j} g_2^{b_j}$ for $j = 1, 2, \cdots, t$. Set $\pi_{share} := (c_0, c_1, \ldots, c_t)$; Sends share (f_i, r_i) securely to party P_i and return π_{share} as the proof.
- $\mathsf{Verify}(n, t, \{f_i, r_i\}_{i=1}^{n}, \pi_{share}) \rightarrow$ `true/false`: Given n, threshold value t, the shares $\{f_i, r_i\}_{i=1}^{n}$, and the threshold proof $\pi_{Share} := (c_0, c_1, \ldots, c_t)$: each party P_i uses his/her share (f_i, r_i) and checks if

$$g_1^{f_i} g_2^{r_i} = \prod_{j=0}^{t} c_j^{i^j}$$

  and outputs either `true` or `false`. The rest of the verification is similar to the Feldman scheme.